A Survey of Small Language Models for Agentic AI: Architectures, Compression, Deployment, and Evaluation

MOHAMMED GAMAL RAGAB*, Universiti Teknologi PETRONAS, Malaysia
HITHAM ALHUSSIAN*, Universiti Teknologi PETRONAS, Malaysia
SAID JADID ABDULKADIR, Universiti Teknologi PETRONAS, Malaysia
W. ELTAYEB AHMED, Imam Mohammad Ibn Saud Islamic University (IMSIU), Saudi Arabia

Large language models (LLMs) fit poorly, on cost, memory, and latency, with a growing class of workloads: AI agents that issue many narrow, repetitive, well-specified model calls. Small language models (SLMs) have narrowed the capability gap through distillation, curated and synthetic data, and quantization. This survey defines an SLM as an autoregressive decoder-only model of at most about twenty billion parameters both active per token, which bounds compute, and in total, which bounds storage; models meeting only the first bound are treated separately as small-active-parameter mixtures of experts. We catalog 179 SLMs and 30 such mixtures released from 2023 to September 2026, check every record's developer source, and release the inventory, the checks, and the scripts that apply the inclusion rule. We review compression along separate axes of parameters, memory, training compute, and latency. We quantify why the vendor evaluation record cannot support cross-model comparison: for the 14 cohort models of the four largest vendors, no benchmark is reported for all of them and no stated protocol is shared by more than five. We then state the agentic case as a conditional, falsifiable hypothesis: for call types that are bounded, recurrent enough to clear a break-even volume counting retries, checks, and escalations, short-context, and checkable for correctness, a specialized SLM should be the default, with escalation to a frontier model when a check fails. Grading the agent-specific evidence, we find per-call competence reported for specialized sub-10B models but qualified by independent studies, contested evidence on end-to-end success, and no measurement of cost per completed task for an SLM-first agent. We state the workloads for which the hypothesis fails and specify a pre-registered comparison, with its decision rule and sample size, that would refute it for a given workload.

CCS Concepts: • **Computing methodologies** → **Natural language processing**; *Neural networks*; *Multi-agent systems*; • **Computer systems organization** → *Embedded and cyber-physical systems.*

Additional Key Words and Phrases: Small language models, agentic AI, model compression, edge AI, on-device inference, LLM–SLM collaboration, knowledge distillation, quantization, efficient NLP

1 Introduction

Large language models (LLMs) such as GPT-4 [94], Claude [6], Gemini [37], and Llama [77] show broad competence across reasoning, code, and multilingual tasks, at a cost that scaling laws [52, 58] made predictable but not small: multi-thousand-GPU training runs, tens of gigabytes of accelerator memory per served replica, and a network round trip, a per-token charge, and a transfer of prompt content for every call. Training GPT-3 is estimated to have consumed 1,287 MWh [98]. Per-query serving energy is contested: a 2023 estimate put a frontier query at about ten times a web search

*Corresponding authors.

Authors' Contact Information: Mohammed Gamal Ragab, mohammed.ragab@utp.edu.my, Department of Computer and Information Sciences, Center for Research in Data Science (CeRDaS), Universiti Teknologi PETRONAS, Seri Iskandar, Perak, Malaysia; Hitham Alhussian, seddig.alhussian@utp.edu.my, Department of Computer and Information Sciences, Center for Research in Data Science (CeRDaS), Universiti Teknologi PETRONAS, Seri Iskandar, Perak, Malaysia; Said Jadid Abdulkadir, saidjadid.a@utp.edu.my, Department of Computer and Information Sciences, Center for Research in Data Science (CeRDaS), Universiti Teknologi PETRONAS, Seri Iskandar, Perak, Malaysia; W. Eltayeb Ahmed, waahmed@imamu.edu.sa, Department of Mathematics and Statistics, College of Science, Imam Mohammad Ibn Saud Islamic University (IMSIU), Riyadh, Saudi Arabia.

2026. ACM 1557-7341/2026/10-ART
https://doi.org/

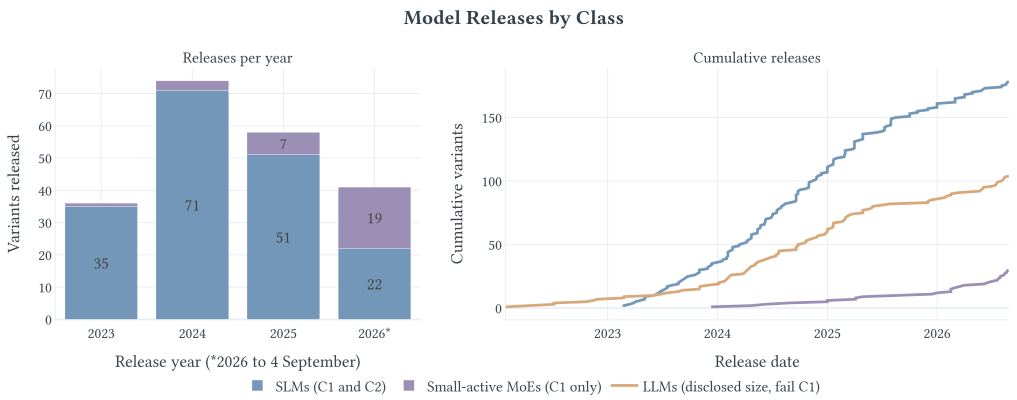


Fig. 1. Documented variants by class: releases per year (left; 2026 to 4 September) and cumulative releases (right), from version 3.6 of the released inventory. SLMs meet both C1 and C2; small-active-parameter MoEs meet C1 only and are plotted separately; the comparison curve counts the 104 records of disclosed size that fail C1. The 119 records of undisclosed size cannot be classified and are not plotted.

[23], while a 2025 production measurement reports 0.24 Wh for the median text prompt of one large assistant [30].

1.1 The Rise of Small Language Models

Against this trajectory, small language models (SLMs) have narrowed the capability gap at fixed size. The Chinchilla analysis [52] showed that most large models were undertrained; Phi showed that curated and synthetic "textbook-quality" data lets a 3.8B model rival much larger ones [1]; Gemma distilled from larger teachers [38]; and Qwen2.5 trained sub-10B models on 18 trillion tokens [102]. The 2025–2026 releases add hybrid state-space architectures [113], controllable reasoning [122], and mobile-first multimodal families [7, 40]. Figure 1 shows the record of the released inventory (Supplementary Section S18): 179 SLMs and, counted separately, 30 small-active-parameter mixtures of experts (both defined below) released between 2023 and 4 September 2026. The release rate of SLMs proper peaked in 2024 (71 variants, against 51 in 2025 and 22 in 2026 to date); what grew in 2026 is the small-active-parameter class (19 of the 41 releases of that year), which Section 1.2 keeps apart because it does not share the deployment properties of an SLM. The seed sources of the inventory changed in 2026 (Section 1.5), so year-to-year differences partly reflect coverage.

1.2 Definitions and Terminology

"Small" is used in the literature to mean at least four different things, so we fix the terms before using them.

1.2.1 The Operative Definition. A model is a *small language model* in this survey when it satisfies both of the following:

C1 (compute envelope) It is an autoregressive decoder-only model (attention, state-space, or a hybrid) whose *active* parameter count per generated token is at most roughly twenty billion.

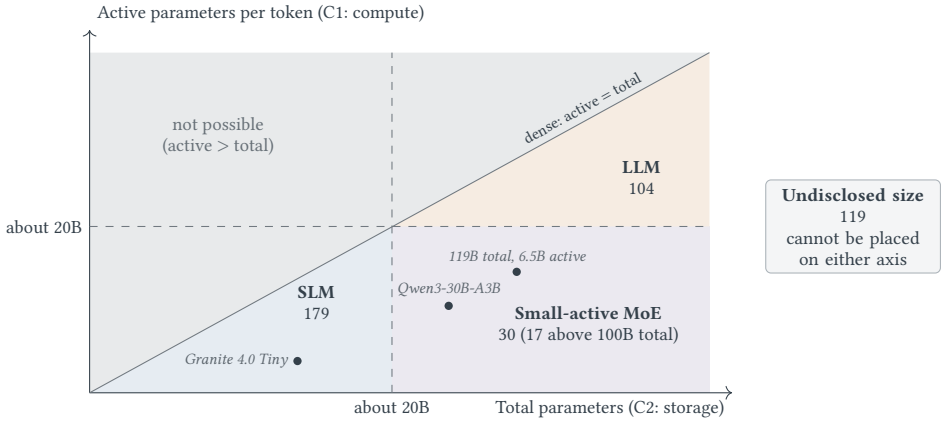


Fig. 2. The two bounds of the definition. C1 bounds active parameters per token, which set per-token compute, latency, and energy; C2 bounds total parameters, which set storage and download size (and resident memory when every weight is kept in memory). Dense models lie on the diagonal. Counts are inventory records (version 3.6); example points are illustrative placements on schematic logarithmic axes, not a plot of the inventory.

C2 (storage envelope) Its *total* parameter count is also at most roughly twenty billion, so that its weights, about 11–15 GB at the 4.4–5.8 effective bits per weight of common 4-bit formats (Supplementary Section S9.5), can be stored and downloaded like a consumer application and, on runtimes that keep every weight in memory, held on one 16–24 GB consumer GPU or laptop.

C1 bounds per-token compute, latency, and energy (Figure 2). C2 bounds storage and download size, and resident memory only on runtimes that keep all weights resident: runtimes that stream weights or experts from flash [4], as Apple's 20B-total, 1–4B-active on-device model does [8], decouple residency from total size, so for such models resident memory has to be reported separately rather than read off C2. The two conditions coincide for dense models, which is why a single threshold has sufficed in most prior work, but they come apart for mixture of experts (MoE) models: a 119B-total model that activates 6.5B parameters per token has the per-token cost of a 6.5B model and the storage cost of a 119B one. The twenty-billion figure is a convention, not a natural kind. Applied to the records of disclosed size in the released file, a bound of 10B, 13B, 20B, or 32B total parameters would admit 143, 160, 179, or 209 of them respectively, and every record carries its counts so that any other threshold can be applied.

1.2.2 Four Distinguished Terms.

SLM A model satisfying C1 and C2. Every deployment, storage, and on-device claim in this paper applies to this class.

Compact model An SLM at or below roughly 3B total parameters, whose 4-bit weights fit in about 2 GB of the 4–8 GB a current phone leaves to applications. Most documented phone deployments use this band; the exceptions keep part of the model off the accelerator, through per-layer embeddings (Gemma 3n and Gemma 4 E-series) or flash-resident experts (Apple's third-generation on-device model).

Edge model A model, of any size, for which the developer documents and ships an artifact for phone-, single-board-computer-, or neural processing unit (NPU)-class hardware. "Edge" is a claim about the deployment target, not about parameter count.

Small-active-parameter MoE A model satisfying C1 but not C2. These have SLM-like per-token cost and LLM-like storage cost. They are recorded in the inventory but counted, plotted, and discussed separately, and are excluded from every claim about storage, download size, resident memory, or on-device feasibility. The class is wide: 17 of its 30 members exceed 100B total parameters, so it includes frontier-scale models such as Llama 4 Maverick and Qwen3.5-397B-A17B as well as 21–35B models.

The inventory holds 179 SLMs and 30 small-active-parameter MoEs; wherever this paper counts SLMs, it means the 179. Per-layer-embedding "effective" sizes (Gemma 3n, Gemma 4 E2B/E4B) are not sparse activation, so C1 is evaluated on those models' total counts. The unit counted is the *variant*: a separately released artifact of one size within a family, which may be a distinct post-training or domain specialization released as its own repository (for example Phi-4 and Phi-4-reasoning), with checkpoint revisions of the same artifact counted once. Counted instead by distinct family and size, the 179 SLMs are 136 family–size pairs. Supplementary Section S1 lists the reporting dimensions the inventory records and their coverage (total parameters 209/209, context length 135/209, peak runtime memory not reported by vendors), and the artifact vocabulary.

1.3 The Agentic Turn and the Thesis

A large share of model calls is now issued by AI agents: control loops in which a model is called repeatedly to choose a tool, fill its arguments, extract a field, classify an intent, or check a result [11, 20]. Many such calls are narrow, repetitive, and format-constrained. Belcak et al. [11] argue that for these calls SLMs should be the default, and Sharma and Mehta [106] survey the evidence and propose an SLM-default, LLM-fallback design. We state the position as a conditional hypothesis whose terms can be measured.

H (default hypothesis). Consider a type of language-model call inside an agent with the following properties.

H1 (bounded) The output is a tool selection, a record conforming to a fixed schema, a label from a fixed set, or a local edit of a few lines.

H2 (recurrent) The expected number of such calls over the planning horizon, N, exceeds the break-even volume

$$N^{*} = \frac{F}{c_L - (a\, c_S + c_V + e\, c_L)}, \tag{1}$$

where F is the one-time and maintenance cost of specializing for the call type, c_L and c_S are the per-call serving costs of the large and the small model, $a \geq 1$ is the expected number of small-model attempts per call, c_V is the per-call cost of the check, and e is the escalation rate. If the denominator is not positive, specialization never pays.

H3 (short-context) The input needed fits within a few thousand tokens without retrieval over a large corpus.

H4 (checkable) Two checks exist at a cost small relative to c_L: a *validity* check (H4a: schema validation, parsing, type checking), and a *correctness* check (H4b: an executor with observable effects, a unit test, a constraint solver, or a lookup against a reference) that detects outputs which are well formed but wrong. Validity checking alone does not satisfy H4.

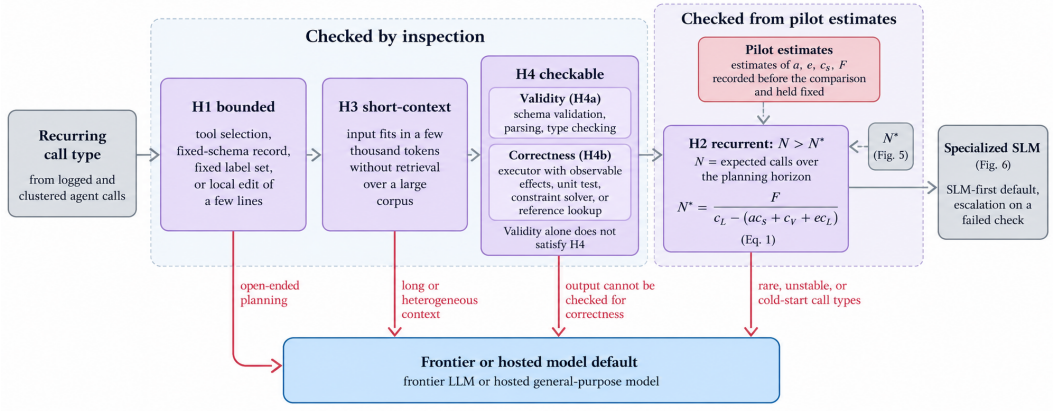


Fig. 3. Assessing one recurring agent call type against hypothesis H at design time. H1, H3, and H4 are checked by inspecting the call type; H2 is checked against pilot estimates recorded before any comparison. A call type that passes every condition defaults to a specialized SLM with escalation on a failed check (Figure 6); a failure at any condition, labelled with the workload class that typically fails there (Section 7.4), leaves a frontier or hosted model as the default.

> For such a call type, a specialized SLM is the appropriate default: the call goes to the SLM first and is escalated to a frontier LLM when a check fails. "Appropriate default" means that the SLM-first system is non-inferior to an all-LLM system in task success, within a margin δ fixed in advance, at a lower cost per completed task than every comparator of Section 9.3.

H1, H3, and H4 can be checked by inspecting a call type before any model is trained (Figure 3). H2 cannot be checked from first principles, because a, e, and c_S depend on the specialized model: it is assessed from pilot estimates recorded before the comparison and held fixed, so that a failed comparison cannot be reclassified afterwards as a failure of H2. H is refuted for a workload class meeting H1–H4 if a pre-registered comparison of the kind specified in Section 9.3 finds the SLM-first system inferior by more than δ, or more costly per completed task than any comparator, and it is rejected as a general design rule if it fails on a majority of at least three pre-registered workloads with distinct call types. It does not claim that SLMs match frontier models in general, that agents can dispense with large models, or any particular share of agent traffic meeting H1–H4. Section 7.4 sets out the workload classes for which the conditions fail. Section 7.3 grades the evidence bearing on H: it is strongest for per-call competence of specialized sub-10B models, contested for end-to-end success, and absent for cost per completed task in an SLM-first agent.

1.4 Scope and Contributions

Existing SLM surveys [48, 79, 91, 116] provide taxonomies, technique reviews, and measurements but treat agentic deployment only in passing. The agentic case has been made as a position [11, 114] and as a survey [106], which already proposes SLM-default, LLM-fallback systems with verifier cascades, cost per successful task as the deciding metric, and limits where fallback remains necessary. We share that direction. This paper adds the following:

A two-bound definition that separates per-token cost (C1) from storage cost (C2) and so treats mixtures of experts consistently (Section 1.2).

A verified inventory of 179 SLMs and 30 small-active-parameter MoEs (2023–2026), in which every record's developer source has been checked and every inclusion decision is recorded with its reason, released with the scripts that apply the rule and generate every table and figure (Supplementary Section S2).

An audit of the vendor evaluation record that quantifies why a cross-model table cannot be built from it: for the 14 cohort models of the four largest vendors, no benchmark is reported for all of them, and no stated protocol is shared by more than five (Section 5).

A compression review that reports reductions along separate axes of parameters, memory, training compute, and latency (Section 4).

An operational version of the agentic hypothesis with a break-even condition that includes retries, checking, and escalation, a correctness requirement on the check, graded agent-specific evidence including contrary results, and the workload classes for which it fails (Section 7).

A falsification design that specifies comparators, the statistical decision rule, and the sample size needed to decide H for a workload, with five further verification studies (Section 9).

1.5 Survey Method and Sources

This is a structured narrative survey, not a systematic review; its aims are representative coverage of the design space, a verified inventory, and an audit of a defined cohort. It addresses five research questions: the design space of SLMs released 2023–2026 (RQ1); which compression techniques pay in the sub-10B regime, along which axis, and at what quality cost (RQ2); what the vendor evaluation record can and cannot support (RQ3); what agent-specific evidence says about small models (RQ4); and under which conditions a small model is the appropriate default inside an agent (RQ5).

Model inventory (RQ1). A seed list compiled in March 2026 from public model listings and aggregators, 205 of whose records were compiled with the assistance of a large language model, was updated on 5 September 2026 against the OpenRouter catalogue and vendor announcements, and every record was then classified against C1 and C2. On 8 October 2026 every record's developer source was checked through the Hugging Face Hub API: 197 of the 209 included records link an existing repository owned by the developer, and the other 12 link the developer's own announcement, paper, or code repository. Twelve records first sourced from an aggregator or a press article were re-sourced to the developer, and one for which no developer source exists was excluded. C1, C2, and inclusion criteria M1 (a developer source states size and architecture) and M4 (a dated release) are computed from recorded fields by a released script; the exclusions of derivative fine-tunes, API aliases, and encoder-decoder models are named editorial decisions listed in that script. The released file records, per record, the check, its date, and the Hub parameter count where available. Coverage is strongest for developers that publish on the Hub. Supplementary Section S2 gives the criteria, the seven excluded records, and the count history.

Technique literature (RQ2, RQ4, RQ5). Works were identified from the prior surveys cited above, by backward and forward citation tracing, and by targeted searches of arXiv and the ACL Anthology, last run on 8 October 2026 for agent-specific evaluation, routing, function calling, and cost-per-outcome measurement. Screening decisions were not recorded for this stream, so statements that we "found no" study are bounded by that search.

Evaluation record (RQ3). For each model of the 18-model cohort, the primary release document and official model card were consulted; the coverage ledger of Section 5.2 was transcribed from them on 8 October 2026. No benchmark was executed for this survey.

Appraisal. Each empirical claim carries an evidence grade (Section 5.1.2) reflecting its design and the independence of its source.

1.6 Organization

Section 2 reviews related work. Section 3 presents the taxonomy, Section 4 compression, Section 5 the evaluation record and the grading scheme, and Section 6 deployment. Section 7 assesses H on agent-specific evidence and states its counterexamples. Section 8 discusses implications, applications, and limitations, Section 9 sets out open problems and the verification agenda, and Section 10 concludes. The electronic supplement (Supplementary Section S1 to Supplementary Section S18) holds the reporting dimensions, inclusion criteria and released artifacts, detailed technique reviews, the audit ledger, reference tables, domain applications, and the generated variant inventory.

2 Related Work and Research Gaps

2.1 From Scaling to Small Models

The Transformer [115] and the pre-train-then-adapt paradigm [14, 27] led to a scaling race that the Chinchilla study [52] partly redirected: a 70B model trained on 1.4 trillion tokens outperformed the 280B Gopher, implying that most large models were undertrained and that smaller, longer-trained models could match them. Subsequent families pushed this further at small scale through curated and synthetic data [1, 46], distillation from larger teachers [38, 39], and pruning plus distillation from a larger sibling, as for Llama 3.2's 1B and 3B edge models [85]. Three bodies of technique underlie the current generation: distillation and synthetic-data training [46, 51]; post-training compression and parameter-efficient adaptation [26, 36, 53, 71]; and on-device runtimes [42, 83, 88]. Sections 4 and 6 review them, and Supplementary Section S3 and S4 summarize the principal families and the longer background.

2.2 LLM–SLM Collaboration and Agentic Systems

A line of work studies how small and large models cooperate [18, 22]. Cascades and routers are the most relevant to agents: FrugalGPT [19] learns a cascade over commercial APIs, Hybrid LLM [28] routes by predicted difficulty, AutoMix [2] escalates on self-verification, RouteLLM [93] learns from preference data, RouterBench [54] benchmarks routers on 405K inference outcomes, and Division-of-Thoughts [105] routes sub-questions between an on-device and a cloud model. Speculative decoding [17, 65] uses a small model to accelerate a large one without delegating answers. On the agentic side, function-calling data pipelines report specialized sub-10B models among the leading entries of the Berkeley Function-Calling Leaderboard [73, 75, 124], and agent evaluation has begun to report cost alongside accuracy [32, 59, 60].

2.3 Gaps in Existing Literature

General SLM surveys [79, 91, 116] provide technique taxonomies, application coverage, and in the case of Lu et al. [79] a measurement study of 70 models; Gupta et al. [48], updated in 2026, survey about 160 papers on 1–8B models. None separates active from total parameters or foregrounds agentic deployment. Srivastava et al. [110] benchmark 72 small models on 17 reasoning benchmarks with three repeats, including models derived by quantization, pruning, and distillation, and find that reasoning tracks training method and data more than scale and that quantization preserves reasoning while pruning disrupts it; it is the closest thing in the literature to the controlled evaluation whose absence Section 5 documents, but covers reasoning only. Feng et al. [34] survey efficient reasoning, and Chen et al. [20] survey multi-agent systems by application.

Two works state the agentic case directly. Belcak et al. [11] argue it as a position, with a conversion algorithm and estimates of how much agent traffic small models could handle, and Tian et al. [114] argue that personal agents must move to the edge. The closest prior work is the survey of Sharma and Mehta [106]. It reviews 1–12B (occasionally about 20B) models for agentic workloads, proposes

Table 1. Coverage of this survey relative to prior surveys and position papers (Y full, P partial, – absent), as judged by the present authors from each work's text (grade D). *Active/total*: separates active from total parameters. *Evaluation critique*: analysis of whether the reporting record supports cross-model comparison. *Graded evidence*: an explicit strength-of-evidence grade per claim. *Agentic focus*: small models inside agents as a primary topic. *Decision rule*: a stated condition under which the position is refuted.

Work	Taxonomy	Compression	Active/ total	Evaluation critique	Graded evidence	Agentic focus	Decision rule
Lu et al. [79]	Y	P	–	P	–	–	–
Nguyen et al. [91]	Y	Y	–	–	–	–	–
Wang et al. [116]	Y	Y	–	–	–	P	–
Gupta et al. [48]	Y	P	–	–	–	–	–
Srivastava et al. [110]	P	P	–	Y	–	–	–
Belcak et al. [11]	–	–	–	P	–	Y	–
Tian et al. [114]	–	–	–	–	–	Y	P
Sharma and Mehta [106]	P	P	–	–	–	Y	–
This survey	Y	Y	Y	Y	Y	Y	Y

an SLM-default, LLM-fallback architecture with uncertainty-aware routing, retries, and verifier repair (their Section VIII), proposes cost per successful task as the deciding metric, and names where large models still win: open-domain synthesis, complex multi-hop reasoning, safety-critical judgment, and long-context work (their Section XI). We share that design and metric. It does not separate active from total parameters, grade its evidence or report contrary results, examine whether the vendor evaluation record supports comparison, or state a decision rule under which its position would be refuted, and its estimate that cost per successful task falls 10–30× (their Section X) is not a reported measurement. Those are the points on which this paper differs: the C1/C2 definition, the verified inventory, the quantified audit of the evaluation record, graded evidence including contrary results, a break-even condition that includes retries, checking, and escalation together with a correctness requirement on the check, and a falsification design with comparators, a decision rule, and a sample size. Table 1 summarizes the positioning.

3 A Faceted Taxonomy of Small Language Models

3.1 The Scheme

We index the SLM design space along four *primary* facets, architecture, training strategy, modality, and application domain (Figure 4), and five *secondary* facets that govern adoption: licensing, data transparency, safety alignment, reasoning mode, and deployment target (Table S9 in Supplementary Section S10). The facets are *separately observable*, in that each can be read off a model's primary source, but they are *not orthogonal*: pruning-plus-distillation presupposes a dense teacher, and a vision encoder ties modality to architecture. The scheme is also *multi-label*: Gemma 3 is distilled *and* trained on a curated corpus, Granite 4.0 is a hybrid state-space model *and* a mixture of experts. It is an indexing scheme illustrated with exemplars, not a partition of the inventory. The released inventory records per variant the fields its sources state directly (modality, access, license, dense or mixture-of-experts, post-training status, context length), and the figures are drawn from those fields; architecture and training facets are assigned in this paper to representative models only. The application-domain facet is the weakest of the four, since it usually describes use rather than the artifact, so we assign it only when the developer trained the released artifact for a domain and says so.

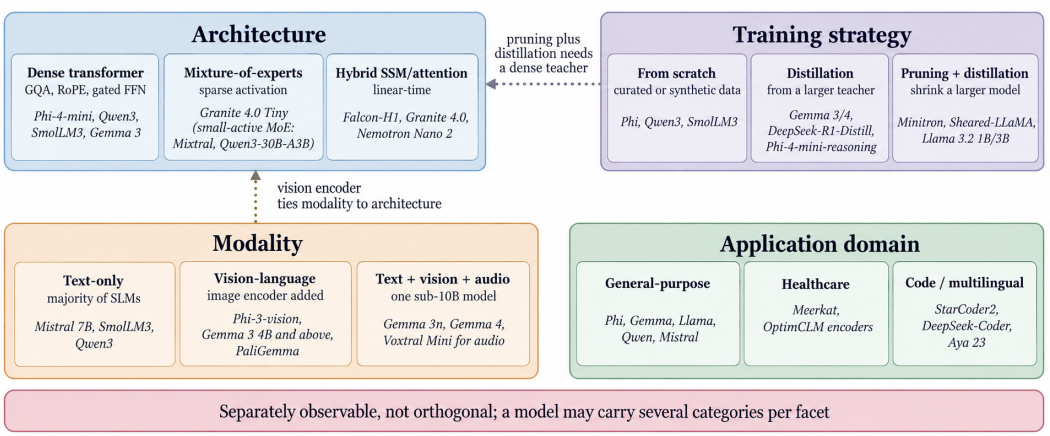


Fig. 4. The four primary facets of the taxonomy. Facets are separately observable but not orthogonal, and assignment is multi-label. Representative models are italicised; Supplementary Section S6 develops each facet and Table S7 gives the tabular form.

3.2 Architecture

Dense transformers remain the majority, from SmolLM2-135M [5] to 7–14B models. The recurring refinements are grouped-query attention [3], which shrinks the key–value (KV) cache by the ratio of query to KV heads; rotary position embeddings with an enlarged base or rescaling for long context; and gated feed-forward layers (SwiGLU in most families, GeGLU in Gemma). Among SLMs whose records state a context length, the median grew from 2K tokens in 2023 ($n = 5$) to 16K in 2024 ($n = 39$) and 128K in 2025 ($n = 46$) and 2026 ($n = 20$); coverage is thin for 2023, so the starting point is uncertain (Supplementary Section S5).

Mixtures of experts [33, 107] decouple total from active parameters, and they sit on the fault line that C2 marks. Qwen3-30B-A3B [122] costs about what a 3B dense model costs per token but needs 17–22 GB of 4-bit weight storage, so it is a small-active-parameter MoE in this paper, not an SLM; Granite 4.0 Tiny (about 7B total, 1B active) [56] satisfies both conditions. The 2026 releases concentrated in this class, with Nemotron 3.5 Lightning (30B/3B), Laguna XS 2.1 (33B/3B), Ling 3.0 Flash (124B/5.1B), and DeepSeek V4 Flash (284B/13B) appearing between June and August 2026.

Hybrid state-space models combine attention with linear-time state-space layers [44]. Jamba [70] interleaves Mamba and attention blocks and reports a KV cache eight times smaller than Mixtral's at 256K context; Falcon-H1 [113], Granite 4.0 [56], Nemotron Nano 2 [92], and the Qwen3.5 small series [103] brought the design to production at small scale in 2025–2026. Supplementary Section S6 gives the architectural detail for each class.

3.3 Training Strategy, Modality, and Domain

Three training routes recur and are often combined: training from scratch on curated or synthetic data (Phi, SmolLM, Qwen), distillation from a larger teacher (Gemma, the DeepSeek-R1 students, Phi-4-mini-reasoning), and pruning a larger model followed by distillation (Minitron, Sheared-LLaMA, Llama 3.2 1B and 3B). Text-only models still dominate the inventory, vision–language models are common at 2–12B, and audio and natively multimodal mobile models (Qwen2.5-Omni, Gemma 3n and 4, Voxtral Mini) grew in 2025–2026 (Supplementary Section S5). Most SLMs are released as

Table 2. What each compression technique reduces, by axis. Entries follow from how the technique works, not from measurements. "Weight memory" is the realized footprint on a runtime that exploits the format; "training compute" is the cost of producing the compressed artifact relative to training an equivalent model from scratch; "latency" is decode-time speedup on memory-bandwidth-bound hardware. A dash means the technique does not act on that axis.

Technique	Parameters	Weight memory	Training compute	Decode latency	Retraining
PTQ, 8-bit	–	2× vs. FP16	–	In proportion to bandwidth saved	No
PTQ, 4-bit	–	2.8–3.6× (Table S3)	–	In proportion to bandwidth saved, batch 1	No
PTQ, 2-bit	–	Below the nominal 8× by the share kept at higher precision	–	As above	quantization-aware training (QAT) in practice
Ternary weights (1.58-bit)	–	About 10× nominal; trained in the format [80]	–	Needs kernels for the format	Trained from scratch
Unstructured pruning, 50%	2×	Only on sparse-capable runtimes	–	None on dense kernels	Calibration only
Structured pruning	Set by target	Proportional to parameters removed	–	Proportional to parameters removed	To recover quality
Knowledge distillation	Set by student	Proportional to student size	Large; the student is trained	Proportional to student size	Student training
Prune + distill	Set by target	As for structured pruning	Up to 40× fewer tokens per derived model [90]	As for structured pruning	Yes

general-purpose models; developer-trained domain models concentrate in code, healthcare, and multilingual use. Supplementary Section S6 lists representative models for each category.

4 Model Compression Techniques for SLMs

Distillation, quantization, and pruning are the principal routes from a large or full-precision model to one deployable on edge hardware, and compound pipelines combine them. This section states how their results are reported here, summarizes what each technique delivers, and grades what the literature establishes about its quality cost. Supplementary Section S7 and S8 review the individual methods.

4.1 How Compression Results Are Reported Here

Compression results are routinely summarized as one "compression ratio" against one "accuracy retention" percentage. That summary does not survive scrutiny. *There is no single compression ratio*: quantization reduces bits per weight and leaves the parameter count untouched; pruning reduces the parameter count, and memory only where the runtime exploits the sparsity pattern; distillation reduces the training compute needed to reach a given quality, while the student's size is the practitioner's choice, so a "10–100×" distillation ratio describes design choices, not a property of the method. *There is no baseline-free retention*: a figure such as "about 96%" is uninterpretable without the baseline precision, model, size, task, metric, and aggregation rule, each of which changes it materially [71, 110]. Table 2 therefore separates what each technique reduces along four axes that are not interchangeable and are not summed, and Table 3 states what the literature establishes about quality cost as a direction with its scope and grade. Where no study supports a number, none is given, and Section 9.3 specifies the measurement that would.

Table 3. What the reviewed literature establishes about the quality cost of each technique in the sub-10B regime, with its scope. Grades follow Section 5.1.2.

Technique	What the literature supports	Scope of the evidence	Grade
PTQ, 8-bit	Effectively lossless on the knowledge and language-modeling benchmarks reported	Method authors' evaluations [25, 120]	C
PTQ, 4-bit	Reasoning accuracy largely preserved in an independent benchmark of small models; quantization methods tuned on large models transfer imperfectly to sub-1B models, with sensitivity depending on architecture	72 models, 17 reasoning benchmarks, three repeats [110]; two models of 0.14B and 0.5B [117]	B
PTQ, 4-bit, by method	activation-aware weight quantization (AWQ) retains more accuracy than GPTQ at matched bit-width	Author-reported [71]	C
PTQ, 2-bit	post-training quantization (PTQ) degrades unacceptably; QAT recovers part of the loss	Vendor pipeline [7]	C
Unstructured pruning, 50–60%	One-shot pruning with negligible perplexity increase at 175B	OPT-175B, BLOOM-176B [35]	C
Quantization vs. post-training pruning	Quantization (GPTQ, AWQ) preserves fidelity and reasoning better than post-training pruning (SparseGPT, Wanda); the advantage shrinks on knowledge-and-reasoning tasks	6 models, 0.5–3.8B, 7 languages, 7 tasks [127]; corroborated across 72 models [110]	A
Prune + distill	Pruning Nemotron-4 15B to 8B and 4B and distilling on under 3% of the original data needs up to 40× fewer training tokens per derived model (1.8× less compute for the family)	One pipeline, one family [90, 109]	C
Domain compound pipeline	Distilling clinical BERT encoders with pruning and 8-bit quantization gives up to 22.88× compression with under 5% macro-AUROC loss	One study, encoder classifiers [49]	C
Fairness under compression	Degradation is uneven across languages; pruning is worse than quantization on multilingual tasks	Two sources [13, 127]	D

4.2 Distillation, Quantization, and Pruning

Knowledge distillation [51] trains a student to match a teacher's output distribution, its intermediate representations (white-box), or its generated text and reasoning traces (black-box), as in Orca [89] and the DeepSeek-R1 students [24]; synthetic-data training, the mechanism behind Phi [46], is distillation through the data. *Quantization* reduces numerical precision. Post-training methods need only a small calibration set: GPTQ [36] uses second-order information, AWQ [71] protects activation-salient channels, and LLM.int8() and SmoothQuant handle activation outliers at 8 bits [25, 120]. GPT-generated unified format (GGUF) [42] is the format in which most sub-4B models are distributed for CPU deployment. Quantization-aware training recovers part of what post-training quantization loses at 2–3 bits but needs the training stack, which is why it appears mainly in vendor pipelines [7], and its limit case, ternary weights trained from scratch [80], is a training recipe rather than a compression of an existing model. *Pruning* removes individual weights (SparseGPT [35], Wanda [111]) or whole structures (LLM-Pruner [81], ShortGPT [82]); structured pruning followed by continued training is a cheaper route to a small model than training from scratch [119].

The one controlled comparison in the sub-10B regime concerns the choice between the first two kinds of post-training compression. Zhou et al. [127] compare GPTQ and AWQ against SparseGPT and Wanda on six models of 0.5–3.8B across seven languages and seven tasks, and find that quantization preserves fidelity, multilingual perplexity, and reasoning accuracy better than pruning, with the advantage shrinking on knowledge-and-reasoning tasks such as OpenBookQA and with Wanda's parity with SparseGPT at large scale not holding at this size; Srivastava et al. [110], across 72 small models, likewise find that quantization preserves reasoning while pruning disrupts it. This is the one grade-A claim in the paper. Its scope is post-training methods at their usual operating points, 4-bit weights against 50% sparsity, which are not equal reductions: 4-bit weights cut memory

by about 3× on any runtime, while 50% unstructured sparsity saves memory only on runtimes that exploit it. The practical reading is that to compress an existing small model after training, quantize first; structured pruning followed by distillation, which derives a new model, is outside the comparison.

4.3 Combined Pipelines

The strongest reported results combine stages. Minitron [90] prunes Nemotron-4 15B in depth and width and retrains the pruned students by distillation from the original on a small fraction of its data; deriving the 8B and 4B models this way needs up to 40× fewer training tokens per model than training from scratch, a 1.8× compute saving for the family, and the derived models are reported to compare with similarly sized community models (C). A follow-up applies the recipe to Llama 3.1 8B and Mistral NeMo 12B [109], and Meta derived Llama 3.2 1B and 3B by pruning Llama 3.1 8B and distilling from the 8B and 70B models [85]. Quantization is applied afterwards for deployment and is not part of these recipes. In clinical NLP, OptimCLM [49] composes distillation, pruning, and 8-bit quantization of BERT encoders (C); it shows that the stages compose, not that they do for generative models.

5 The Public Evaluation Record for Small Language Models

This section examines what the public evaluation record can support about SLM capability. The values discussed are *reported* figures from vendor technical reports, model cards, community leaderboards, and third-party studies. We set out the cohort, sources, and grading scheme (Section 5.1), quantify why a cross-model table cannot be assembled from vendor reporting (Section 5.2), and explain why these benchmarks do not answer the agentic question in any case (Section 5.3).

5.1 Cohort, Sources, and Grading

5.1.1 Cohort and Sources. The cohort was chosen purposively: 18 models with public weights or an evaluation card, below 10B parameters (Phi-4 at 14B retained as an upper-bound reference), at least one score on an established benchmark, release no later than March 2025 so that at least eighteen months of third-party evaluation had accumulated by the snapshot, and a spread of training methods (Table S4 in Supplementary Section S10). It spans 0.5B to 14B and eight developers, and it contains none of the 2025–2026 hybrid, reasoning, or mobile-multimodal families, so the audit describes reporting practice for 2023–2025 releases. Values were sought from vendor technical reports, then official model cards, then single-harness community evaluations such as HELM [69] and the Open LLM Leaderboard [55], then peer-reviewed third-party studies. A reported number is comparable with another only if five conditions match: benchmark version, prompt template, answer-extraction rule, decoding settings, and harness [12]; Supplementary Section S9 defines each.

5.1.2 Evidence Grading. Because this survey mixes controlled studies, vendor self-reports, independent evaluations, and our own synthesis, each empirical claim carries a grade. The grade is set by two properties of the strongest source behind the claim: its *design* (a controlled comparison, in which the compared conditions differ only in the variable of interest under one harness with a matched baseline; a single-harness evaluation of many systems; a self-report; or an estimate or synthesis) and its *independence* (whether the reporting party has a stake in the result).

A **Controlled, independent** A controlled comparison reported by a party without a stake in the outcome.

B **Independent** Any other evaluation or documented count by a party without a stake: a single-harness evaluation across models, an independent benchmark, or a reproduction.

Table 4. Evidence grade of the principal claims of this survey (A controlled and independent, B independent, C reported by the interested party, D interpretation or estimate). "None" marks a quantity for which no study was located by the search of Section 1.5.

Claim	Grade	Where
For the 14 cohort models of the four largest vendors, no benchmark is reported for all, and no stated protocol is shared by more than five (released ledger)	B	§5.2
Reasoning ability of small models tracks training method and data more than scale	B	§2.3
The strongest 3–4B models of 2024–2025 report scores that 13B-class models of 2023 did not (across sources and time)	D	§5.2
INT8 post-training quantization is effectively lossless on the benchmarks its method authors report	C	§4
At 4 bits, reasoning is largely preserved; methods tuned on large models transfer imperfectly to sub-1B models	B	§4
At 0.5–3.8B, post-training quantization preserves fidelity better than post-training pruning	A	§4.2
4-bit weight footprint is 2.8–3.6× below FP16 (arithmetic from parameter count and effective bits)	D	§5.4
Specialized sub-10B function-calling models placed among the leading single-turn entries in 2024	C	§7.3.1
Specialized small models are reported ahead of frontier models on multi-turn function calling and τ-bench	C	§7.3.1
Without task-specific fine-tuning, small models often fail the required call format	B	§7.3.1
Serving a 7B model is 10–30× cheaper in latency, energy, and FLOPs than a 70–175B model	D	§7.2.2
40–70% of the LLM calls in three open-source agents could be handled by specialized SLMs	D	§7.1
End-to-end success on realistic agent benchmarks was low for all models when the benchmarks were introduced	B	§7.3.3
Open models of 70B or less trailed commercial APIs end to end (2023; size confounded with openness and training)	B	§7.3.3
Current specialized sub-10B agents trail frontier agents end to end	D	§7.3.3
Cost per correct answer favors lightweight models on basic quantitative tasks and large models on knowledge-intensive ones (single-turn)	B	§7.3.4
Cost per completed task, SLM-first agent against all-LLM agent on one workload	None	§7.3.4
Hypothesis H	D	§1.3

C **Reported** Stated by the developer of the artifact or the author of the technique, including developer-run comparisons on a single harness.

D **Interpretation** Our synthesis across sources, an inference from evidence gathered for another question, or an estimate in a position paper.

Table 4 grades the principal claims, and Tables 3 and 5 grade the technique and agent evidence. A grade-C claim is unverified rather than false; where vendor reports and independent reproductions of the same model exist, the vendor figure is usually the more favorable, so the likely direction of error is known.

5.2 Why a Common Benchmark Table Cannot Be Built from Vendor Reporting

We originally intended a single table placing the cohort side by side on a shared benchmark set, assembled from the vendors' documents. The vendor record cannot support it, and we report the measured reasons instead. The released coverage ledger (`audit_coverage_ledger.csv`; Supplementary Section S9.2) records, for the 14 cohort models from Microsoft, Alibaba, Meta, and Google, whether each model's primary release document reports an instruction-tuned value on

each of the eight benchmarks that recur in their headline reporting (Massive Multitask Language Understanding (MMLU), MMLU-Pro, ARC-Challenge, HellaSwag, Grade School Math 8K (GSM8K), MATH, HumanEval, MBPP), and under which stated protocol.

Vendors do not report a common benchmark set. Of the 112 model–benchmark cells, 69 carry an instruction-tuned value, and no benchmark is reported for all 14 models; the best covered, GSM8K and MATH, are reported for 11.

Where they report the same benchmark, they rarely use the same protocol. Only 45 of the 69 reported cells state a protocol at all; the Qwen2.5 report, for example, gives no shot counts for its instruction-tuned tables. Among stated protocols, no benchmark has a protocol shared by more than five of the 14 models: MMLU is 5-shot for Phi and Llama 3.2 and 0-shot for Gemma 3; GSM8K is 8-shot with chain-of-thought for Phi and Llama 3.2 and 0-shot for Gemma 3; ARC-Challenge is 10-shot for Phi and 0-shot for Llama 3.2. Shot count alone moves multiple-choice accuracy by several points [12], so a column headed by one protocol would be false for most of its cells. This finding needs no rule beyond protocol matching.

Pre-trained and instruction-tuned checkpoints are reported separately. For 10 of the 14 models, the vendor tabulates the pre-trained and the instruction-tuned checkpoint in different tables under different protocols, and secondary sources often copy a figure without its checkpoint. For Gemma 3-4B, the technical report gives GSM8K 38.4 for the pre-trained checkpoint (8-shot) and 89.2 for the instruction-tuned one (0-shot with chain-of-thought), and MMLU-Pro 29.2 and 43.6 respectively [39]; a row mixing the two misstates a gap by tens of points.

Matched comparisons exist but are vendor-run. Some vendors publish single-harness comparison tables: Microsoft compares Phi-4-mini with ten other models, five of them in this cohort, on an internal platform that keeps prompts and few-shot examples identical across models [87], and Meta reports Llama 3.2 1B and 3B at BF16 beside their quantized versions on the same harness [84]. These meet the harness condition but are run by an interested party that also chose the comparators and benchmarks, so they are grade C evidence and cannot be merged across vendors.

The general point, that scores are not comparable across evaluation setups, is the premise of HELM, the Open LLM Leaderboard, and the reproducibility analysis of Biderman et al. [12]; the contribution here is to quantify it for the SLM cohort. Single-harness third-party evaluations are the right basis for comparison: the Open LLM Leaderboard v2 ran every submission through one configuration on six benchmarks [55], Lu et al. [79] evaluate 70 models of 0.1–5B under one setup including on-device latency, and Srivastava et al. [110] evaluate 72 models under one reasoning protocol. None covers the whole cohort, and their benchmark sets differ from the vendors', so they replace the vendor table rather than filling it in. What can be said from the vendor record without distortion is qualitative (D): the strongest 3–4B models of 2024–2025 report scores that 13B-class models of 2023 did not, and no single SLM leads across all task families.

5.3 Why These Benchmarks Do Not Answer the Agentic Question

Every benchmark above measures a model answering a question in isolation. An agent asks a model to select one of several tools, to emit arguments that satisfy a schema, to stay well formed across a long tail of inputs, to recover when a tool returns an error, and to do so repeatedly. A model can score well on MMLU and still emit malformed calls often enough to be unusable in a loop, and a small model tuned for function calling can score poorly on MMLU and be adequate as an executor. Even an internally consistent version of this record would therefore be the wrong evidence for H, and Section 7.3 assembles agent-specific evidence instead.

5.4 Quantization Impact

Four points summarize the compression evidence for the cohort's size range (Table 3): INT8 post-training quantization is reported as effectively lossless (C); at 4 bits, reasoning is largely preserved in an independent benchmark of 72 small models, while methods tuned on large models transfer imperfectly to sub-1B models (B); AWQ is reported to retain more accuracy than GPTQ (C); and post-training quantization preserves fidelity better than post-training pruning at 0.5–3.8B (A). Per-model degradation figures are not reported consistently enough to tabulate. The memory consequence is arithmetic: 4-bit formats store 4.4–5.8 effective bits per weight, so weights shrink 2.8–3.6× rather than 4×; Qwen2.5-7B falls from 15.2 GB to 4.2–4.5 GB (Table S3 in Supplementary Section S9.5).

5.5 Multilingual and Safety Reporting

Multilingual and safety reporting is sparser than benchmark reporting and not comparable across models. Models whose reports document large non-English pretraining mixtures, such as Qwen2.5 [102], Gemma 3 [39], and Aya 23 8B [9], report stronger non-English performance than English-centric families (C). Per-model toxicity evaluations are often absent for research-grade small models, alignment can be eroded by fine-tuning [100], and compression degrades performance unevenly across languages [13, 127]. An agent that routes non-English or safety-sensitive calls to a small English-centric model inherits these gaps, which argues for capability-aware rather than size-only routing.

6 Deployment on Resource-Constrained Environments

6.1 On-Device Deployment and What It Guarantees

On-device inference removes the network round trip, the per-token charge, and the transmission of the prompt to an inference provider, and it works offline. It does not by itself keep user data on the device. Whether data leaves depends on whether the application emits telemetry or crash reports containing prompt content, logs conversations to a synchronized store, escalates calls to a cloud model (in which case, under difficulty-based routing, the escalated calls may be exactly the sensitive ones), or sends the same content to a hosted retrieval index or remote tool. On-device inference removes one egress channel, and eliminates egress only in an application whose whole pipeline is local and whose telemetry is absent or content-free. Apple's design [7] treats the escalation path as the hard part and builds Private Cloud Compute around it. Every privacy claim in this paper is subject to this scope condition.

6.1.1 Platforms and Device Tiers. Apple's Foundation Models are the most fully documented on-device deployment: the 2025 generation pairs a roughly 3B on-device model, quantized with 2-bit quantization-aware training and KV-cache sharing, with a server model on Private Cloud Compute [7], and the 2026 generation adds a 20B-total on-device model that activates 1–4B parameters and keeps its full weights in flash rather than DRAM [8]. Apple describes the 2026 family as built in collaboration with Google [8], following a joint statement that its next foundation models would be based on Gemini models and cloud technology [43]. Gemini Nano plays the on-device role on Android [37]. A current phone leaves roughly 4–8 GB to applications, so a 3B model at 4 bits (about 2 GB) fits, subject to thermal throttling and battery cost. Single-board computers and Jetson-class modules run quantized 1–7B models, and microcontrollers are out of reach for any model in the inventory (Supplementary Section S11). Table S8 (Supplementary Section S10) compares the main runtimes: llama.cpp, ONNX Runtime, MLX, TensorRT-LLM, ExecuTorch, MLC-LLM, and WebLLM.

6.2 Calculated Quantities Versus Measured Ones

Weight memory at a precision, KV-cache size per token, and their ratio follow from parameter count, bit-width, and head configuration, so they are reliable within their assumptions (Table S3). Throughput, time-to-first-token, sustained rates under thermal load, energy per token, battery cost, and total cost of ownership are measured quantities that depend on batch size, context length, kernels, achieved bandwidth, runtime version, thermal envelope, and for energy the grid mix and the amortization of the device. Published figures for them are not comparable across sources, and we do not tabulate them; where one appears, it is attributed to its source and hardware.

One widely repeated claim needs particular care: that quantizing from 16 to 4 bits gives a near-linear 4× speedup. It holds only for single-stream decoding at batch size 1 on memory-bandwidth-bound hardware, where every generated token reads the full weight set. Under batching the bottleneck moves toward compute, during the compute-bound prefill phase the gain largely disappears, on hardware without native low-precision kernels dequantization consumes part of it, and the memory saving is itself 2.8–3.6× rather than 4×. Meta's measurements on a OnePlus 12 phone show the size of the shortfall: 4-bit Llama 3.2 1B and 3B decode 2.6× faster than their BF16 versions, and their resident memory falls by 40–50% [84] (C). The defensible statement is that 4-bit quantization converts a storage saving into a decode-time saving in proportion to the bandwidth it frees, and that single-stream interactive use, the dominant on-device regime, is where it does so.

Two further constraints bind on the edge. Prefill is compute-bound and sets time-to-first-token, which grows with context length; decode is bandwidth-bound and sets tokens per second. Grouped-query attention (GQA) and local–global attention shrink the KV cache that every decode step reads, which is why they matter more on a device than on a server. For a bounded call that a small model completes correctly, these properties make the small model cheaper per call and faster once the network hop is counted; whether it is cheaper per completed task, once retries, checks, and escalations are counted, is the question Section 7 takes up.

7 When Should a Small Model Be the Default in an Agent?

The preceding sections show that SLMs can be compressed to run on commodity and edge hardware at a characterized cost (Sections 4–6), and that the general benchmark record can establish neither their relative capability nor their fitness for agents (Section 5). This section assesses hypothesis H (Section 1.3) on agent-specific evidence. The premises and the conversion pipeline are those of Belcak et al. [11], and the SLM-default, LLM-fallback design with cost per successful task as its metric is shared with Sharma and Mehta [106]; what this section adds is the grading of each line of evidence, including contrary results, the break-even and checking conditions of H, and the counterexamples of Section 7.4.

7.1 What Agents Delegate

An agent is a control loop in which language models operate alongside tools, memory, and a scaffold of prompts and parsers. The model is mostly asked to perform bounded sub-tasks: choose a tool and its arguments, extract fields, classify an intent, rewrite a query, summarize a passage, produce a short patch, or check a step. Belcak et al. [11] characterize such calls as narrow, repetitive, and format-constrained, and estimate, by inspecting three open-source agents, that about 60% of the LLM queries in MetaGPT, 40% in Open Operator, and 70% in Cradle could be handled by appropriately specialized SLMs. These are the authors' judgments of replaceability, not measurements and not an application of H1–H4, so we grade them D. We found no trace study measuring the distribution of call types in deployed agents by count, cost, or contribution to task success.

7.2 The Premises

7.2.1 P1: Capability Sufficiency. The first premise is that a specialized SLM is good enough for the individual bounded call, not for the completed trajectory. Section 7.3.1 sets out the evidence, which is favorable for tool selection and argument filling but mostly developer-reported. Two further results bear on it. Controllable reasoning extends what can be delegated: Qwen3 [122] exposes a thinking budget, and distilled reasoning models such as Phi-4-mini-reasoning [121] report competition-mathematics results at 3–8B that earlier models of that size did not reach (C). And specialization, not size alone, appears to carry the gain: a controlled study of Qwen3 models from 0.6B to 32B, published after our snapshot, finds supervised fine-tuning with low-rank adaptation (LoRA) the strongest in-distribution adaptation for tool calling across the whole range [64] (B). All of the per-call evidence concerns models below 10B or a single family.

7.2.2 P2: Economics. Belcak et al. [11] estimate that serving a 7B SLM is 10–30× cheaper in latency, energy, and FLOPs than serving a 70–175B LLM, and Sharma and Mehta [106] estimate that cost per successful task falls by a similar factor; both are estimates rather than measurements (D). The break-even condition H2 (Equation 1, Figure 5) is how we state the premise instead, because four things that a per-token ratio omits decide the outcome. First, the unit is cost per *completed task*: a small model that needs a attempts per call, a check costing c_V, and escalation of a fraction e of calls to the large model saves only $c_L - (a\,c_S + c_V + e\,c_L)$ per call, and saves nothing once $e \geq 1-(a\,c_S+c_V)/c_L$. Second, specialization has fixed costs F: logging and curating traces, removing personal data, fine-tuning per call type, evaluation, router calibration, retraining under drift, and maintaining a fleet of specialists. Third, the large-model cost c_L is not a list price: agent calls repeat long system prompts and tool schemas, which is the case that provider-side prompt caching and asynchronous batch pricing discount, and hosted fine-tuning of a provider's small model gives specialization without a fleet. Each of these lowers c_L or F for the comparator and so raises N^*, as does any further fall in hosted prices at fixed capability. Fourth, the comparator for server-side calls is not only a frontier model: a hosted general-purpose small model or a small-active-parameter MoE has small-model per-token cost and no specialization cost. The one measurement of cost per correct answer across model classes finds that lightweight models are cost-optimal on basic quantitative tasks and large models on knowledge-intensive ones [32] (B), which supports a conditional premise and not a general one. Finally, "zero marginal cost" on a device means no per-query charge to the operator, not zero cost to a user who pays in battery and thermal budget; offline, the large model cannot run at all, which makes the question one of capability rather than cost.

7.2.3 P3: Control. An agent needs its model to emit exactly the format the scaffold expects. Constrained decoding enforces well-formedness outside the model [29, 118], so syntactic validity for a supported schema does not depend on model size. This removes an objection to small models rather than giving them an advantage, since a large model can be constrained equally. What remains for P3 is the argument of Belcak et al. [11] that a narrow output distribution is easier to impose on a small model by fine-tuning on the agent's own traces than to elicit from a frontier model by prompting, together with operational properties: small models are easier to version, roll back, host on-premises, and co-locate with their tools.

7.3 Agent-Specific Evidence

General benchmark scores are the wrong instrument for these premises (Section 5.3). This subsection assembles the evidence that bears on them, by the capability an agent exercises, and grades each line (Table 5).

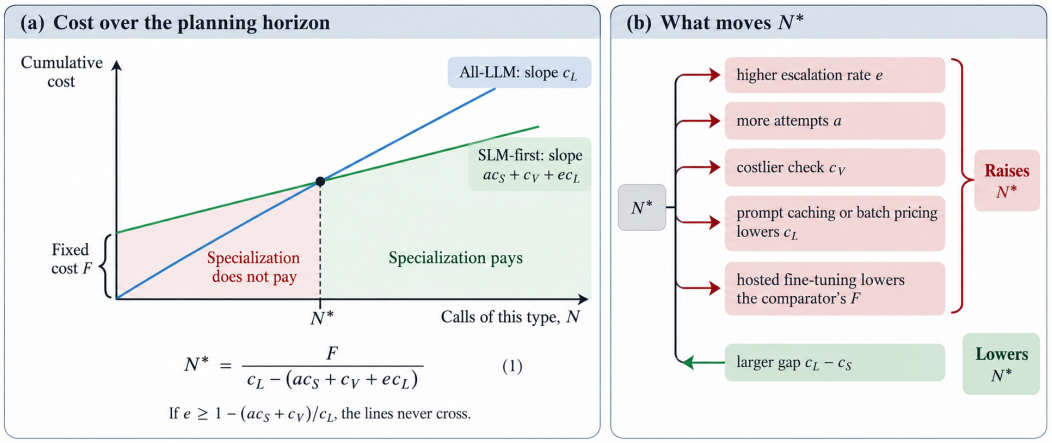


Fig. 5. Break-even volume for specializing one call type (Equation 1). (a) An SLM-first system pays a fixed cost F and a per-call cost that includes attempts, the check, and escalations, so it is cheaper than an all-LLM system only above N^*; if $e \geq 1 - (a\,c_S + c_V)/c_L$, the lines never cross. (b) Factors that move N^*. Schematic: no axis values or prices are implied, and no measured N^* exists for any workload.

7.3.1 Function Calling and Tool Selection. This is the best-evidenced capability. The Berkeley Function-Calling Leaderboard [96] checks function selection and argument filling with executable checks, in single-turn, parallel, and, from its third version, multi-turn settings; its predecessors [67, 97, 101] established that API-call generation is a skill on which general capability transfers only partially. Developer reports place specialized sub-10B models among the leading single-turn entries of 2024: APIGen's 7B model was reported to outperform several GPT-4 models and its 1B model GPT-3.5-Turbo [75]; the xLAM-7B card places it third at 88.24% (the first place belonged to an 8×22B member of the series) [124]; ToolACE reports an 8B model rivalling GPT-4 [73]; and on-device work reports 0.1–7B models competitive on narrower suites [21, 31, 72, 76]. Meta reports Llama 3.2 3B at 67.0 on BFCL v2 against 67.1 for Llama 3.1 8B, and the 1B model at 25.7 [84]. For multi-turn use, APIGen-MT trains 1B–70B models on verified multi-turn trajectories and reports that they outperform GPT-4o and Claude 3.5 on τ-bench and BFCL, with smaller models surpassing larger ones in multi-turn settings and more consistent across repeated trials [99]. All of these are developer reports on public benchmarks, and the placements were not checked against archived leaderboard snapshots (C).

Independent evidence qualifies them. Kavathekar et al. [61] find that small models improve from zero- to few-shot to fine-tuned function calling but "struggle significantly with adhering to the given output format" (B), and Shen et al. [108] recover performance only by splitting planning, calling, and summarization across separately fine-tuned small models. Read together, the evidence supports the conditional form of H: per-call competence at this size is something specialization produces, not something an off-the-shelf small model has.

7.3.2 Structured Output: Validity Is Not Correctness. Constrained decoding guarantees that output parses, at an overhead its authors report as small [29, 118] (C). Three results limit what that buys. JSONSchemaBench finds that frameworks differ in schema coverage, efficiency, and output quality, so the guarantee holds for the features a framework supports [41] (B). Grammar-constrained decoding distorts the model's distribution, producing grammatical outputs whose likelihoods are not those the model assigns [95]. And Tam et al. [112] report that format restrictions degrade reasoning

(B), a result disputed by the developers of a constrained-decoding library on methodological grounds [63], so its size is uncertain. What constrained decoding does not do is make the content correct, and the gap between well-formed and correct output, a valid call to the wrong tool or with a wrong argument value, is where small models still lose ground. This is why H4 requires a correctness check (H4b) and not only a validity check (H4a).

7.3.3 End-to-End Agent Success. Here the evidence is weakest and partly contradictory. When the realistic multi-step benchmarks were introduced, success was low for every model: the best GPT-4 agent reached 14.41% on WebArena against 78.24% for humans [126], GPT-4 with plugins 15% on GAIA against 92% [86], and the best model about 2% on SWE-bench [57] (B). The one cross-class comparison of that period, AgentBench, found a large gap between commercial models and open models of 70B or less, attributed to long-horizon reasoning, decision-making, and instruction following [74] (B); it compared open with closed models of 2023, so model size is confounded with openness and training. Against it, the APIGen-MT developers report specialized small models ahead of frontier models on τ-bench [99] (C). We found no dated, independent, like-for-like comparison of current specialized sub-10B agents and frontier agents, so whether a size gap persists for specialized small models is open (D).

The failure mode these benchmarks expose is the one per-call metrics hide. τ-bench reports that even strong models succeed on under half of its tasks and degrade as the number of repeated trials grows [123], and τ^2-bench reports further drops when the agent must coordinate with an active user [10]. A trajectory of n calls at per-call reliability p completes at p^n absent recovery, so the difference between 0.98 and 0.95 per call is the difference between 82% and 60% over ten steps. A correctness check that triggers escalation (H4b) is what breaks this compounding; a validity check alone lets valid but wrong outputs pass and compound.

7.3.4 Routing, Escalation, and Cost per Completed Task. Cascades and routers report large savings on single-turn query benchmarks: FrugalGPT matches the best single model with up to 98% cost reduction [19], Hybrid LLM sends up to 40% fewer calls to the large model with no quality drop [28], AutoMix reports over 50% cost reduction at comparable quality [2], and RouteLLM over 2× at matched quality with transfer to unseen model pairs [93] (all C). RouterBench provides a common dataset of 405K inference outcomes for comparing routers [54]. None of these reports, inside an agent loop, how often the router is confidently wrong, how escalation rates move under distribution shift, or what the seam costs when a trajectory switches models.

Cost per correct outcome has begun to be measured, but not for the comparison H requires. Cost-of-pass, the expected monetary cost of a correct solution, has been measured across model classes on single-turn tasks [32] (B); the Holistic Agent Leaderboard reports agent accuracy and cost across 9 frontier-class models and 9 benchmarks, and finds, for example, that higher reasoning effort reduced accuracy in most runs [59] (B); and Kapoor et al. [60] argue that agents should be evaluated on joint cost and accuracy. We found no study that reports cost per completed task for an SLM-first agent against an all-LLM agent on the same workload. That is the measurement on which H is confirmed or refuted (Section 9.3).

7.3.5 What This Evidence Supports. The evidence supports a narrower claim than the one usually made. Developers report (C), and independent work qualifies (B), that a *specialized* sub-10B model can perform the individual delegated operation at a standard competitive with much larger models; an off-the-shelf small model often cannot. Validity can be guaranteed independently of model size for supported schemas; correctness cannot. Whether a specialized small-model agent completes realistic multi-step tasks at a comparable rate is contested, and whether it does so at lower cost per

Table 5. Agent-specific evidence bearing on hypothesis H, by capability, with the premise or condition each line bears on. Grades follow Section 5.1.2; "None" marks a quantity for which the search of Section 1.5 located no study. Per-call evidence is favorable but mostly developer-reported, independent evidence qualifies it, and the completed-task comparison H requires is absent.

Capability	What the evidence shows	Source class	Bears on	Grade
Single-turn function calling	Specialized sub-10B models among the leading entries in 2024	Developer reports [73, 75, 84, 124]	P1, H1	C
Multi-turn function calling	Specialized small models reported ahead of frontier models on BFCL multi-turn and τ-bench	Developer report [99]	P1	C
Format adherence without fine-tuning	Small models often fail the call format; fine-tuning helps most	Independent study [61]	P1	B
Specialization method	LoRA fine-tuning strongest in-distribution for tool calling at 0.6–32B	Independent study [64]	P1	B
Structured-output validity	Constrained decoding guarantees parseable output, model-independently	Method papers [29, 118]	H4a	C
Limits of constrained decoding	Coverage differs by framework; decoding distorts the distribution; reasoning effect disputed	[41, 63, 95, 112]	H4	B
Multi-step reliability	Success degrades over repeated trials of the same task	Independent benchmarks [10, 123]	H4b	B
End-to-end success	Low for all models at introduction; open ≤70B models trailed commercial APIs in 2023	Independent benchmarks [57, 74, 86, 126]	P1	B
Current size gap end to end	Contested: no independent like-for-like comparison of specialized sub-10B and frontier agents	–	P1	D
Estimated replaceable calls	40–70% of calls in three agents judged replaceable by specialized SLMs	Position paper [11]	H2	D
Router quality	Cascades and routers cut cost at matched quality on single-turn benchmarks	Author-reported [2, 19, 28, 93]	P2	C
Router calibration, escalation rate	Not reported inside agent loops	–	P2, H2	None
Cost per correct outcome	Lightweight models cost-optimal on basic quantitative tasks, large ones on knowledge-intensive tasks (single-turn); agent cost and accuracy for frontier models	Independent [32, 59]	P2	B
Cost per completed task, SLM-first vs. all-LLM	Not reported	–	H	None

completed task has not been measured. The case for H is therefore strongest where a correctness check exists and the call type recurs often enough to repay specialization, and unproven elsewhere.

7.4 Where the Hypothesis Does Not Hold

For the following workload classes the conditions of H fail and a frontier model remains the right default.

Open-ended planning When the agent must decide *what* to do, the call fails H1: there is no fixed schema and the space of acceptable plans is large. AgentBench locates the open-model gap in long-horizon reasoning and decision-making [74], and a planning error propagates to every later step.

Synthesis over long or heterogeneous context H3 fails for large document sets, long trajectory histories, and multi-source retrieval requiring cross-document reasoning. Long context in a small model is a memory and prefill problem before it is a capability problem, and we found no evidence that small models close the gap on long-context synthesis (D).

Uncheckable output H4b fails for open-ended generation, judgment calls, and tasks where verification is as hard as the task; errors then compound silently.

Low-resource-language and safety-sensitive calls The multilingual and safety deficits of Section 5.5 are concentrated in small models, and a router that sends such calls to a small model on size grounds degrades them silently.

Sparse or unstable call distributions H2 fails when a call type is rare, when traffic drifts faster than specialization can track (raising F), or when cached, batched, or fine-tuned hosted alternatives make c_L low; a hosted model is then cheaper in total.

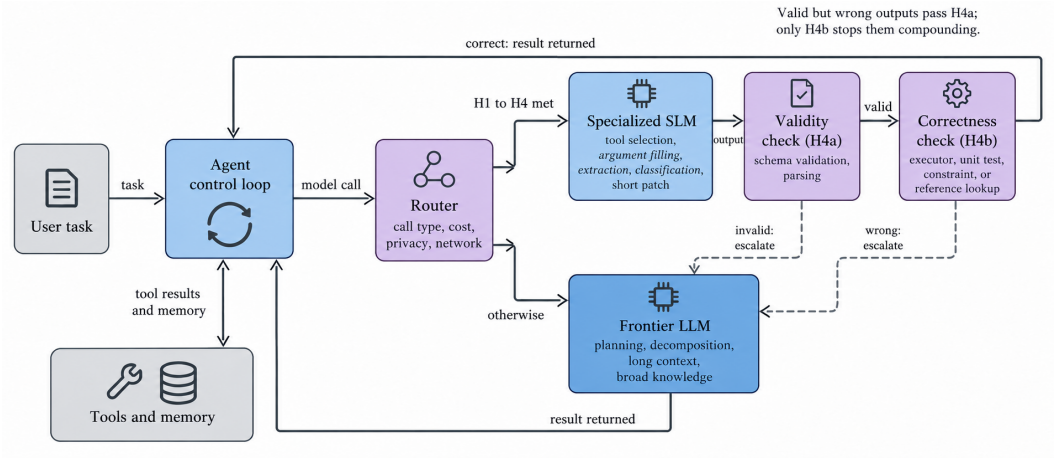


Fig. 6. Heterogeneous SLM–LLM agent. A per-call router sends call types meeting H1–H4 to specialised small models by default; their outputs are checked for validity and correctness, and a failed check escalates the call to a generalist LLM, which also handles planning-like and long-context calls. The split between the paths varies by agent. After Belcak et al. [11].

Cold start Before deployment there are no logged traces for the conversion pipeline. The opening move is a frontier model with instrumentation, and migration afterwards.

Planning and uncheckable generation are not transitional: they are categories of work for which narrow competence and constrained output are the wrong properties, and the heterogeneous design below exists because of them.

7.5 Heterogeneous SLM–LLM Systems

For call types meeting H1–H4 the default inverts: SLMs handle them, and a generalist model handles planning, open-ended decomposition, long context, broad world knowledge, and calls whose small-model output fails a check [11, 106]. Figure 6 sketches the arrangement; the share of calls on each path depends on the agent and is not established in general.

7.5.1 Routing Designs. Three designs recur: cascades that try the small model first and escalate on a quality signal [2, 19], predictive routers that decide before generation [28, 93], and decomposing routers that assign sub-questions to a local or a cloud model [105]. H prescribes a cascade whose quality signal is the H4 check. Speculative decoding [15, 17, 65, 68] speeds up the large model without delegating answers, and an escalated call can itself be served speculatively. Supplementary Section S12.3 describes each.

7.5.2 Production Instances. Apple pairs an on-device model with a server model on Private Cloud Compute and routes to the server when the on-device model is expected to fall short [7]; its 2026 models were built in collaboration with Google [8, 43]. The design is motivated by privacy and latency as much as by cost, and Apple reports no cost per completed task, so it shows that the heterogeneous pattern is deployable at scale, not that it is cheaper. Google's Gemini Nano occupies the same on-device role on Android [37].

7.6 Conversion and Adoption

For an agent that already runs on a frontier model, Belcak et al. [11] propose a six-step conversion: log the agent's calls, strip personal data, cluster them into recurring types, choose a candidate SLM per cluster, specialize it with LoRA or quantized low-rank adaptation (QLoRA), and repeat as traffic drifts. The clustering step is where H1, H3, and H4 can be checked and where the pilot estimates for H2 are made; clusters that fail stay with the generalist, and each converted cluster should be evaluated on cost per completed task against the unconverted agent. They also identify three barriers to adoption: sunk investment in centralized inference, evaluation on generalist leaderboards, and lower attention to SLM releases; Section 7.3 shows the second to be partly a real evidential gap. Supplementary Section S12 gives the pipeline and barriers in full.

8 Discussion

8.1 What the Evidence Supports

Four conclusions follow from the preceding sections, each at the grade stated.

Capability per parameter has risen, and training data is the main lever. The strongest 3–4B models of 2024–2025 report scores that 13B-class models of 2023 did not report, an across-source comparison whose direction alone we assert (D); a single-harness benchmark of 72 small models finds that reasoning tracks training method and data quality more than scale [110] (B).

To compress an existing small model, quantize first. Post-training quantization preserves fidelity better than post-training pruning at 0.5–3.8B (A), 4-bit weights are 2.8–3.6× smaller than FP16, and the decode speedup that follows is real only in single-stream, bandwidth-bound use, as Meta's measured 2.6× on a phone illustrates (Section 6.2). A single accuracy-retention figure per technique is not supported (Section 4.1).

The vendor evaluation record cannot support cross-model comparison. For the 14 cohort models of the four largest vendors, no benchmark is reported for all of them and no stated protocol is shared by more than five (B). Comparison tables built from vendor figures, including those in secondary sources, inherit this; single-harness third-party evaluations are the basis on which comparison should rest.

Right-sizing is a conditional design rule, not a general one. For a call type meeting H1–H4, a specialized small model is plausibly the right default: it is cheaper per call, faster once the network hop is counted, available offline, and its errors are caught by the check. That conclusion is graded D, because the quantity that decides it, cost per completed task against cached, batched, hosted, and frontier alternatives, has not been measured (Section 7.3.4), and Section 7.4 names the workloads where it fails.

8.2 Implications for Agentic Systems

Three implications follow for agent builders. Routing must be capability-aware, not size-aware: a "small model unless hard" router degrades non-English and safety-sensitive calls silently (Section 5.5). The economical unit is often a fine-tuned small model per recurring call type rather than one general SLM, which is what the conversion pipeline produces. And the check is part of the design, not an implementation detail: without a correctness check (H4b), an SLM-first system inherits the compounding of per-call errors that the cascade was meant to stop.

8.3 Domain Applications

The domains where SLM deployment is most advanced are each a workflow of bounded, repeated language operations under regulatory or cost constraints, and Supplementary Section S13 reviews them with the state of the evidence for each. Healthcare is the best evidenced: Meerkat-7B reports

70.6% on MedQA [62] and compressed clinical encoders lose under 5% in macro-AUROC [49] (C), but exam benchmarks are not deployment evidence, and on-premises hosting removes one data-transfer question without satisfying HIPAA or the GDPR. Legal NLP has public benchmarks, including LegalBench [45], LexGLUE [16], and CUAD [50], but we found no evaluation of current sub-10B decoder models across them under one protocol, so its case for small on-premises models is structural rather than evidential. Code has mature small models evaluated on public benchmarks [47, 78, 104], and multilingual coverage trades breadth against depth at this size [9]. In every vertical, deployment interest runs ahead of the evidence.

8.4 Access and Sustainability

Running a capable 3B model on inexpensive hardware, with no per-query charge and, within the scope conditions of Section 6.1, no transfer of prompt content, widens access for organizations facing cloud cost, data-sovereignty rules, or unreliable connectivity. The environmental case is narrower than usually stated. Per-query compute scales with active parameters, but per-query energy also depends on utilization, and a well-batched server can be more efficient per token than an idle device; aggregate effects depend further on teacher training, device manufacture, grid intensity, update distribution, and rebound. We therefore make no aggregate energy claim (Supplementary Section S16).

8.5 Limitations

No original measurement. No benchmark was executed for this survey; the one grade-A claim rests on cited controlled studies, and Section 9.3 is written to be acted on.

Inventory provenance. The inventory began from secondary listings, and 205 records were first compiled with the assistance of a language model. Every included record now links a developer source checked on 8 October 2026, but coverage of models no listing surfaced, especially from smaller developers, is not guaranteed, and the change of seed source in 2026 affects year-to-year comparisons.

Literature search. The technique literature was not screened under a recorded protocol, so statements that no study exists are bounded by the search described in Section 1.5.

Definitional conventions. The twenty-billion threshold, the C1/C2 split, and the sub-10B cohort are conventions; Section 1.2.1 gives the counts under other thresholds.

Scope of H's evidence. Every agent-specific result concerns models below 10B, and H relies on C1 more than C2: for server-side calls a small-active-parameter MoE has small-model per-call cost, so it is a comparator in the test of H rather than part of H.

Coverage. The benchmarks audited and most of the compression literature are English-centric, the audit covers 2023–2025 releases, and the snapshot ages; the update policy of Supplementary Section S2.5 bounds rather than removes this.

8.6 Ethical Considerations

Alignment is harder to preserve at small scale, several research-grade SLMs ship without safety training, and fine-tuning can erode what exists [100]; deployments that give small models tool access should treat output filtering and monitoring as mandatory. Compression can degrade performance unevenly across languages and dialects [13], and pruning more than quantization [127], so high-stakes applications need per-group performance floors. Against these risks, on-device SLMs remove one egress channel, work without connectivity, and need no subscription, which matters most in under-resourced healthcare and education settings.

9 Challenges and Future Directions

9.1 Model-Level Challenges

Hallucination is more pronounced at small scale, where fewer facts fit in the parameters, and retrieval augmentation is the main mitigation [66, 125]. Multi-step reasoning is where the gap to large models is widest; test-time compute, symbolic tools, and reasoning distillation narrow it. Hardware and model architecture are still designed separately, although NPU efficiency depends on matching attention and feed-forward shapes to the accelerator. Compression can amplify bias [13, 127]. Supplementary Section S14 treats each.

9.2 Open Problems for Agentic SLM Systems

Several problems stand between hypothesis H and routine practice. **Correctness checking:** H4b requires a check that catches valid but wrong outputs at a cost well below a large-model call; executors with observable effects and unit tests exist for code and tool calls, but most agent call types lack them. **Difficulty estimation without an oracle:** routers use win prediction [93], predicted difficulty [28], self-verification [2], or decomposition [105], but calibration under distribution shift is unsolved. **Multi-objective routing:** accuracy, latency, energy, cost, privacy sensitivity, and network availability must be weighed jointly, and size-only routing degrades non-English and safety-sensitive calls (Section 5.5). **Handoff without a seam:** escalating mid-task must not produce a visible discontinuity. **Agent-appropriate, cost-aware evaluation:** function-calling and interaction benchmarks [10, 96, 123] and cost-aware agent leaderboards [59] exist, but report specialized sub-10B models sparsely and do not compare SLM-first with all-LLM agents. **Fleet management:** many fine-tuned specialists raise questions of versioning, drift detection, and retraining cadence that a single model does not.

9.3 Empirical-Verification Agenda

Six studies would most reduce the uncertainty documented here. Studies 1–4 use one model set chosen by a stated rule: the instruction-tuned checkpoint of the most downloaded sub-10B variant on the Hugging Face Hub at the time of the study for each of the cohort families (Phi, Qwen, Gemma, Llama) and for the 2025–2026 families (for example Qwen3, Gemma 3n, SmolLM3, Granite 4, LFM2), so that the results bear on both the audited and the current generation. Each fixes the harness version, prompts, extraction rules, and decoding settings in advance, runs sampled tasks with at least three seeds, and reports confidence intervals.

1. **Harness-consistent reruns (RQ3).** Run the model set on the eight benchmarks of the coverage ledger under one `lm-evaluation-harness` configuration, separately for pre-trained and instruction-tuned checkpoints, so that every vendor figure in the ledger has an independent counterpart.
2. **Quantization degradation (RQ2).** Measure FP16, INT8, and three INT4 formats (GGUF Q4_K_M, GPTQ, AWQ) on the same benchmarks with matched baselines, to establish per-model degradation and whether it varies with size.
3. **Edge measurement (RQ2).** Measure decode and prefill throughput, time-to-first-token, resident memory, and energy per token on a phone, a single-board computer, a Jetson-class module, and a laptop, reporting batch size, context length, and runtime version.
4. **Agent-specific evaluation (RQ4).** Run the model set on BFCL multi-turn splits and τ-bench, with and without constrained decoding, reporting pass^k consistency and classifying failures as invalid (caught by H4a) or valid but wrong (caught only by H4b).
5. **Cost per completed task (RQ5, the test of H; Figure 7).** Fix a workload with a checkable success criterion, identify the call types meeting H1, H3, and H4 by inspection, and

record pilot estimates of N, F, a, and e for H2 before the comparison. Run six configurations end to end, each with the same checks and the same retry budget: (i) a frontier model throughout; (ii) the same with prompt caching and batch pricing where the provider offers them; (iii) a hosted general-purpose small model or small-active-parameter MoE without specialization; (iv) a hosted small model fine-tuned through the provider's service; (v) a specialized SLM with escalation to the frontier model on a failed check, which is the configuration H prescribes; and (vi) the specialized SLM without escalation, to measure what escalation buys. Report per configuration task success, calls and retries per completed task, escalation rate, the rate of valid but wrong outputs that pass the checks, latency, and monetary and energy cost per completed task, with F reported separately. The decision rule is a pre-registered one-sided non-inferiority test of (v) against (i) on task success at margin δ, with task-clustered bootstrap intervals, together with a comparison of cost per completed task against every other configuration. For independent arms at a success rate near 0.5, a one-sided test at $\alpha = 0.05$ with power 0.8 needs about $n \approx 3.09/\delta^2$ tasks per arm: about 7,700 for $\delta = 2$ percentage points, 1,240 for 5, and 310 for 10; a paired design on the same tasks needs fewer. A τ-bench-scale workload of a few hundred tasks can therefore decide only a margin of about ten points, and a tighter margin needs a synthetic or production workload of thousands of tasks. Under the refutation rule of Section 1.3, H is rejected for a workload class if (v) fails non-inferiority or costs more per completed task than any other configuration, and rejected as a general rule if it fails on a majority of at least three pre-registered workloads with distinct call types.

6. **Call-type prevalence (H2).** On logged traces of deployed agents, measure the share of calls, of cost, and of contribution to task success that meets H1–H4, replacing the three estimates of Section 7.1 with measurements.

Studies 1–4 are practical on one modern GPU in weeks; studies 5 and 6 need an agent harness, an API budget, and, for study 6, access to production traces. Study 5 is the one that bears directly on the paper's central hypothesis.

9.4 Further Directions

Beyond verification, Supplementary Section S15 sets out multilingual evaluation, on-device continual learning, ensembles of specialized SLMs against a single LLM, multimodal evaluation, linear-time hybrid architectures, and federated fine-tuning.

10 Conclusion

This survey fixed a two-condition definition that separates a model's per-token cost from its storage cost, cataloged 179 SLMs and, separately, 30 small-active-parameter mixtures of experts released from 2023 to September 2026 with every record's developer source checked, reviewed compression along separate axes, and quantified why the vendor evaluation record cannot support cross-model comparison: for the 14 cohort models of the four largest vendors, no benchmark is reported for all of them and no stated protocol is shared by more than five. Single-harness third-party evaluations, not vendor tables, are the basis on which comparison should rest.

On the agentic question, we stated the case for small models as a conditional hypothesis. For call types that are bounded, recurrent enough to clear a break-even volume that counts retries, checks, and escalations, short-context, and checkable for correctness as well as validity, a specialized small model should be the default, with escalation to a frontier model when a check fails. The evidence supports this unevenly. Developers report specialized sub-10B models competitive on single- and multi-turn function calling, while independent work shows that off-the-shelf small

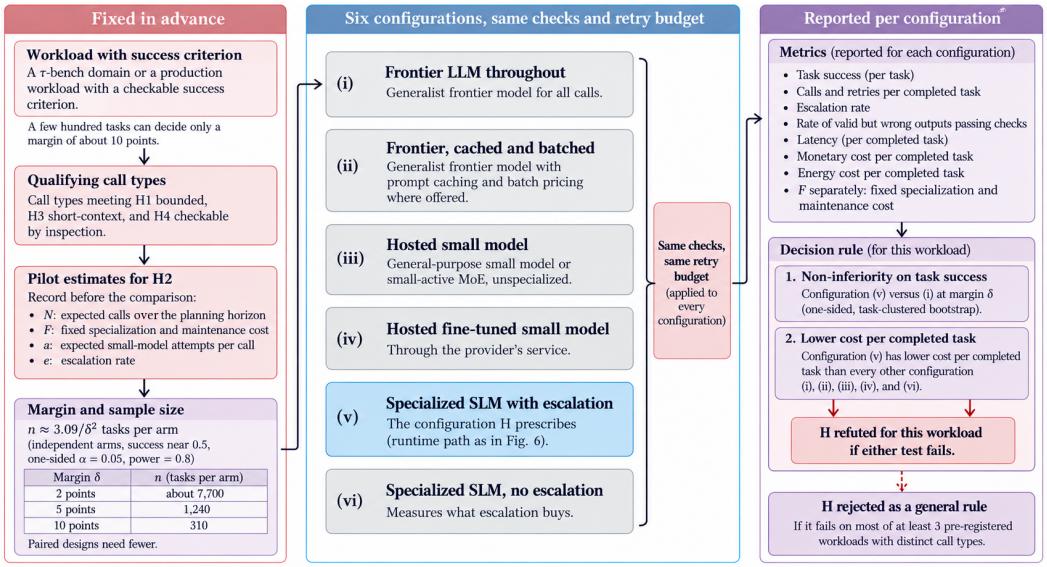


Fig. 7. The pre-registered test of hypothesis H (study 5). Call types, pilot estimates for H2, the margin, and the sample size are fixed before running; six configurations are run end to end under the same checks and retry budget; the decision rule combines a non-inferiority test on task success with a comparison of cost per completed task. The sample sizes assume independent arms at a success rate near 0.5, one-sided $\alpha = 0.05$, and power 0.8. No results are implied: the study has not been run.

models often are not; end-to-end comparisons are contested; cost per correct answer has been measured for single-turn tasks and frontier agents; and no study measures cost per completed task for a small-model-first agent against the alternatives, including cached, batched, and fine-tuned hosted models. Where the conditions fail, for planning, long-context synthesis, and uncheckable work, a frontier model remains the right choice.

The pre-registered comparison of Section 9.3, with its comparators, decision rule, and sample size, is the shortest route from what has been reported to what has been established. Until it is run, the position defended here is a design hypothesis with a stated domain: specialized small models doing the bounded, recurrent, checkable work, and a large model for planning, synthesis, and the calls no cheap check can verify.

Declarations

Funding

This work was supported and funded by the Deanship of Scientific Research at Imam Mohammad Ibn Saud Islamic University (IMSIU), grant number IMSIU-DDRSP2602.

Use of Generative AI

A large language model assisted in compiling 205 of the candidate model records from public listings in March 2026 (Section 1.5). During revision, Anthropic's Claude was used to assist with editing the manuscript, checking bibliographic metadata against arXiv and Crossref records, writing the scripts that check each record's developer source and apply the inclusion rule, and transcribing the coverage ledger from the documents it cites. Every included record's developer source was checked through the Hugging Face Hub API or against the developer's own page, and the authors

reviewed all generated output, verified factual claims and references against their sources, and take full responsibility for the content.

Competing Interests

The authors declare that they have no competing interests.

Ethics Approval

Not applicable: this is a review of publicly available documentation and involves no human participants, human data, or animals.

Data and Code Availability

No benchmark was run for this review; all performance figures are reported values from public sources, whose provenance is given in Supplementary Section S17. The following artifacts are provided as supplementary files with this submission and will be deposited in a versioned archival repository under a persistent identifier on publication: the variant inventory `merged_llm_database.json` (version 3.6), one record per variant with its parameter counts, C1/C2 status, inclusion decision and reason, developer source, the result and date of the source check, and a changelog naming every record added, moved, or removed between versions; the source-check log `source_check_log.csv`; the audit coverage ledger `audit_coverage_ledger.csv`; and the scripts that correct and verify the records, apply the inclusion rule, build the ledger, and generate every inventory table and figure (`fix_records_v3_6.py`, `verify_sources.py`, `apply_inclusion_rules.py`, `build_audit_ledger.py`, `generate_model_tables.py`, `generate_figures_v2.py`).

Author Contributions

M.G.R.: Conceptualization, Methodology, Investigation, Data curation, Software, Writing (original draft), Visualization. **H.A.:** Conceptualization, Methodology, Writing (review and editing), Supervision, Project administration. **S.J.A.:** Investigation, Validation, Writing (review and editing). **W.E.A.:** Validation, Writing (review and editing), Funding acquisition. All authors read and approved the final manuscript.

References

[1] Marah Abdin, Jyoti Aneja, Hany Awadalla, Ahmed Awadallah, Ammar Ahmad Awan, Nguyen Bach, Amit Bahree, Arash Bakhtiari, Jianmin Bao, Harkirat Behl, Alon Benhaim, Misha Bilenko, Johan Bjorck, Sébastien Bubeck, Martin Cai, Qin Cai, Vishrav Chaudhary, Dong Chen, Dongdong Chen, Weizhu Chen, Yen-Chun Chen, Yi-Ling Chen, Hao Cheng, Parul Chopra, Xiyang Dai, Matthew Dixon, Ronen Eldan, Victor Fragoso, Jianfeng Gao, Mei Gao, et al. 2024. Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone. arXiv:2404.14219. arXiv:2404.14219 https://arxiv.org/abs/2404.14219

[2] Pranjal Aggarwal, Aman Madaan, Ankit Anand, Srividya Pranavi Potharaju, Swaroop Mishra, Pei Zhou, Aditya Gupta, Dheeraj Rajagopal, Karthik Kappaganthu, Yiming Yang, Shyam Upadhyay, Manaal Faruqui, and Mausam. 2023. AutoMix: Automatically Mixing Language Models. arXiv:2310.12963. arXiv:2310.12963 https://arxiv.org/abs/2310.12963

[3] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. 2023. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. arXiv:2305.13245. arXiv:2305.13245 https://arxiv.org/abs/2305.13245

[4] Keivan Alizadeh, Iman Mirzadeh, Dmitry Belenko, Karen Khatamifard, Minsik Cho, Carlo C Del Mundo, Mohammad Rastegari, and Mehrdad Farajtabar. 2024. LLM in a flash: Efficient Large Language Model Inference with Limited Memory. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*. arXiv:2312.11514 https://arxiv.org/abs/2312.11514

[5] Loubna Ben Allal, Anton Lozhkov, Elie Bakouch, Gabriel Martín Blázquez, Guilherme Penedo, Lewis Tunstall, Andrés Marafioti, Hynek Kydlíček, Agustín Piqueres Lajarín, Vaibhav Srivastav, Joshua Lochner, Caleb Fahlgren, Xuan-Son

Nguyen, Clémentine Fourrier, Ben Burtenshaw, Hugo Larcher, Haojun Zhao, Cyril Zakka, Mathieu Morlon, Colin Raffel, Leandro von Werra, and Thomas Wolf. 2025. SmolLM2: When Smol Goes Big – Data-Centric Training of a Small Language Model. arXiv:2502.02737. arXiv:2502.02737 https://arxiv.org/abs/2502.02737

[6] Anthropic. 2024. The Claude 3 Model Family: Opus, Sonnet, Haiku. Anthropic technical report. https://www.anthropic.com/news/claude-3-family

[7] Apple. 2025. Apple Intelligence Foundation Language Models: Tech Report 2025. arXiv:2507.13575. arXiv:2507.13575 https://arxiv.org/abs/2507.13575

[8] Apple Machine Learning Research. 2026. Introducing the Third Generation of Apple's Foundation Models. https://machinelearning.apple.com/research/introducing-third-generation-of-apple-foundation-models.

[9] Viraat Aryabumi, John Dang, Dwarak Talupuru, Saurabh Dash, David Cairuz, Hangyu Lin, Bharat Venkitesh, Madeline Smith, Jon Ander Campos, Yi Chern Tan, Kelly Marchisio, Max Bartolo, Sebastian Ruder, Acyr Locatelli, Julia Kreutzer, Nick Frosst, Aidan Gomez, Phil Blunsom, Marzieh Fadaee, Ahmet Üstün, and Sara Hooker. 2024. Aya 23: Open Weight Releases to Further Multilingual Progress. arXiv:2405.15032. arXiv:2405.15032 https://arxiv.org/abs/2405.15032

[10] Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik Narasimhan. 2025. τ^2-Bench: Evaluating Conversational Agents in a Dual-Control Environment. arXiv:2506.07982. arXiv:2506.07982 https://arxiv.org/abs/2506.07982

[11] Peter Belcak, Greg Heinrich, Shizhe Diao, Yonggan Fu, Xin Dong, Saurav Muralidharan, Yingyan Celine Lin, and Pavlo Molchanov. 2025. Small Language Models are the Future of Agentic AI. arXiv:2506.02153. Version 3, 22 September 2026. arXiv:2506.02153 https://arxiv.org/abs/2506.02153

[12] Stella Biderman, Hailey Schoelkopf, Lintang Sutawika, Leo Gao, Jonathan Tow, Baber Abbasi, Alham Fikri Aji, Pawan Sasanka Ammanamanchi, Sidney Black, Jordan Clive, Anthony DiPofi, Julen Etxaniz, Benjamin Fattori, Jessica Zosa Forde, Charles Foster, Jeffrey Hsu, Mimansa Jaiswal, Wilson Y. Lee, Haonan Li, Charles Lovering, Niklas Muennighoff, Ellie Pavlick, Jason Phang, Aviya Skowron, Samson Tan, Xiangru Tang, Kevin A. Wang, Genta Indra Winata, François Yvon, and Andy Zou. 2024. Lessons from the Trenches on Reproducible Evaluation of Language Models. arXiv:2405.14782. arXiv:2405.14782 https://arxiv.org/abs/2405.14782

[13] Su Lin Blodgett, Solon Barocas, Hal Daumé III, and Hanna Wallach. 2020. Language (Technology) is Power: A Critical Survey of "Bias" in NLP. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, Online, 5454–5476. doi:10.18653/v1/2020.acl-main.485

[14] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, et al. 2020. Language Models are Few-Shot Learners. *Advances in Neural Information Processing Systems* 33 (2020), 1877–1901. arXiv:2005.14165 https://arxiv.org/abs/2005.14165

[15] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. 2024. Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads. arXiv:2401.10774. arXiv:2401.10774 https://arxiv.org/abs/2401.10774

[16] Ilias Chalkidis, Abhik Jana, Dirk Hartung, Michael Bommarito, Ion Androutsopoulos, Daniel Martin Katz, and Nikolaos Aletras. 2022. LexGLUE: A Benchmark Dataset for Legal Language Understanding in English. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*. arXiv:2110.00976 https://arxiv.org/abs/2110.00976

[17] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. 2023. Accelerating Large Language Model Decoding with Speculative Sampling. arXiv:2302.01318. arXiv:2302.01318 https://arxiv.org/abs/2302.01318

[18] Lihu Chen and Gaël Varoquaux. 2026. What is the Role of Small Models in the LLM Era: A Survey. *Computational Linguistics* (2026). arXiv:2409.06857 doi:10.1162/coli.a.652

[19] Lingjiao Chen, Matei Zaharia, and James Zou. 2023. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. arXiv:2305.05176. arXiv:2305.05176 https://arxiv.org/abs/2305.05176

[20] Shuaihang Chen, Yuanxing Liu, Wei Han, Weinan Zhang, and Ting Liu. 2024. A Survey on LLM-Based Multi-Agent System: Recent Advances and New Frontiers in Application. arXiv:2412.17481. arXiv:2412.17481 https://arxiv.org/abs/2412.17481

[21] Wei Chen and Zhiyuan Li. 2024. Octopus v2: On-device language model for super agent. arXiv:2404.01744. arXiv:2404.01744 https://arxiv.org/abs/2404.01744

[22] Yi Chen, JiaHao Zhao, and HaoHao Han. 2025. A Survey on Collaborative Mechanisms Between Large and Small Language Models. arXiv:2505.07460. arXiv:2505.07460 https://arxiv.org/abs/2505.07460

[23] Alex de Vries. 2023. The Growing Energy Footprint of Artificial Intelligence. *Joule* 7, 10 (2023), 2191–2194. doi:10.1016/j.joule.2023.09.004

[24] DeepSeek-AI. 2025. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. arXiv:2501.12948. https://arxiv.org/abs/2501.12948

[25] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. 2022. LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale. arXiv:2208.07339. arXiv:2208.07339 https://arxiv.org/abs/2208.07339

[26] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. QLoRA: Efficient Finetuning of Quantized LLMs. In *Advances in Neural Information Processing Systems*, Vol. 36. Curran Associates, Inc., New Orleans, Louisiana, USA. arXiv:2305.14314 doi:10.52202/075280-0441

[27] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. Association for Computational Linguistics, Minneapolis, Minnesota, 4171–4186. doi:10.18653/v1/N19-1423

[28] Dujian Ding, Ankur Mallick, Chi Wang, Robert Sim, Subhabrata Mukherjee, Victor Ruhle, Laks V. S. Lakshmanan, and Ahmed Hassan Awadallah. 2024. Hybrid LLM: Cost-Efficient and Quality-Aware Query Routing. arXiv:2404.14618. arXiv:2404.14618 https://arxiv.org/abs/2404.14618

[29] Yixin Dong, Charlie F. Ruan, Yaxing Cai, Ruihang Lai, Ziyi Xu, Yilong Zhao, and Tianqi Chen. 2024. XGrammar: Flexible and Efficient Structured Generation Engine for Large Language Models. arXiv:2411.15100. arXiv:2411.15100 https://arxiv.org/abs/2411.15100

[30] Cooper Elsworth, Keguo Huang, David Patterson, Ian Schneider, Robert Sedivy, Savannah Goodman, Ben Townsend, Parthasarathy Ranganathan, Jeff Dean, Amin Vahdat, Ben Gomes, and James Manyika. 2025. Measuring the environmental impact of delivering AI at Google Scale. arXiv:2508.15734. arXiv:2508.15734 https://arxiv.org/abs/2508.15734

[31] Lutfi Eren Erdogan, Nicholas Lee, Siddharth Jha, Sehoon Kim, Ryan Tabrizi, Suhong Moon, Coleman Hooper, Gopala Anumanchipalli, Kurt Keutzer, and Amir Gholami. 2024. TinyAgent: Function Calling at the Edge. arXiv:2409.00608. arXiv:2409.00608 https://arxiv.org/abs/2409.00608

[32] Mehmet Hamza Erol, Batu El, Mirac Suzgun, Mert Yuksekgonul, and James Zou. 2025. Cost-of-Pass: An Economic Framework for Evaluating Language Models. arXiv:2504.13359. arXiv:2504.13359 https://arxiv.org/abs/2504.13359

[33] William Fedus, Barret Zoph, and Noam Shazeer. 2022. Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity. *Journal of Machine Learning Research* 23 (2022), 1–39. arXiv:2101.03961 https://arxiv.org/abs/2101.03961

[34] Sicheng Feng, Gongfan Fang, Xinyin Ma, and Xinchao Wang. 2025. Efficient Reasoning Models: A Survey. arXiv:2504.10903. arXiv:2504.10903 https://arxiv.org/abs/2504.10903

[35] Elias Frantar and Dan Alistarh. 2023. SparseGPT: Massive Language Models Can Be Accurately Pruned in One-Shot. In *Proceedings of the 40th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 202)*. PMLR, Honolulu, Hawaii, USA, 10323–10337. https://proceedings.mlr.press/v202/frantar23a.html

[36] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. 2023. GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. In *11th International Conference on Learning Representations (ICLR)*. OpenReview.net, Kigali, Rwanda. https://openreview.net/forum?id=tcbBPnfwxS

[37] Gemini Team, Google. 2023. Gemini: A Family of Highly Capable Multimodal Models. arXiv:2312.11805. arXiv:2312.11805 https://arxiv.org/abs/2312.11805

[38] Gemma Team, Google DeepMind. 2024. Gemma: Open Models Based on Gemini Research and Technology. arXiv:2403.08295. arXiv:2403.08295 https://arxiv.org/abs/2403.08295

[39] Gemma Team, Google DeepMind. 2025. Gemma 3 Technical Report. arXiv:2503.19786. https://arxiv.org/abs/2503.19786

[40] Gemma Team, Google DeepMind. 2026. Gemma 4 Technical Report. arXiv:2607.02770. Google DeepMind; models released 2 April 2026 (E2B–31B, dense and MoE). arXiv:2607.02770 https://arxiv.org/abs/2607.02770

[41] Saibo Geng, Hudson Cooper, Michał Moskal, Samuel Jenkins, Julian Berman, Nathan Ranchin, Robert West, Eric Horvitz, and Harsha Nori. 2025. JSONSchemaBench: A Rigorous Benchmark of Structured Outputs for Language Models. arXiv:2501.10868. arXiv:2501.10868 https://arxiv.org/abs/2501.10868

[42] Georgi Gerganov. 2023. llama.cpp. GitHub repository: https://github.com/ggerganov/llama.cpp.

[43] Google and Apple. 2026. Joint Statement from Google and Apple. https://blog.google/company-news/inside-google/company-announcements/joint-statement-google-apple/. Published 12 January 2026.

[44] Albert Gu and Tri Dao. 2023. Mamba: Linear-Time Sequence Modeling with Selective State Spaces. arXiv:2312.00752. arXiv:2312.00752 https://arxiv.org/abs/2312.00752

[45] Neel Guha, Julian Nyarko, Daniel E. Ho, Christopher Ré, Adam Chilton, Aditya Narayana, Alex Chohlas-Wood, Austin Peters, Brandon Waldon, Daniel N. Rockmore, Diego Zambrano, Dmitry Talisman, Enam Hoque, Faiz Surani, Frank Fagan, Galit Sarfaty, Gregory M. Dickinson, Haggai Porat, Jason Hegland, Jessica Wu, Joe Nudell, Joel Niklaus, John Nay, Jonathan H. Choi, Kevin Tobia, Margaret Hagan, Megan Ma, Michael Livermore, Nikon Rasumov-Rahe, Nils Holzenberger, Noam Kolt, Peter Henderson, Sean Rehaag, Sharad Goel, Shang Gao, Spencer Williams, Sunny

Gandhi, Tom Zur, Varun Iyer, and Zehua Li. 2023. LegalBench: A Collaboratively Built Benchmark for Measuring Legal Reasoning in Large Language Models. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*. arXiv:2308.11462 https://arxiv.org/abs/2308.11462

[46] Suriya Gunasekar, Yi Zhang, Jyoti Aneja, Caio César Teodoro Mendes, Allie Del Giorno, Sivakanth Gopi, Mojan Javaheripi, Piero Kauffmann, Gustavo de Rosa, Olli Saarikivi, Adil Salim, Shital Shah, Harkirat Singh Behl, Xin Wang, Sébastien Bubeck, Ronen Eldan, Adam Tauman Kalai, Yin Tat Lee, and Yuanzhi Li. 2023. Textbooks Are All You Need. arXiv:2306.11644. arXiv:2306.11644 https://arxiv.org/abs/2306.11644

[47] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Y. Wu, Y. K. Li, Fuli Luo, Yingfei Xiong, and Wenfeng Liang. 2024. DeepSeek-Coder: When the Large Language Model Meets Programming – The Rise of Code Intelligence. arXiv:2401.14196. https://arxiv.org/abs/2401.14196

[48] Akanksha Gupta, Bijo Thomas, Harshita Asnani, Phanindra Reddy Madduru, Samia Feroze, Shreyas Subramanian, Vikram Elango, and Mecit Gungor. 2025. Small Language Models (SLMs) Can Still Pack a Punch: A survey (updated 2026). arXiv:2501.05465. arXiv:2501.05465 https://arxiv.org/abs/2501.05465

[49] Mohammad Junayed Hasan, Fuad Rahman, and Nabeel Mohammed. 2025. OptimCLM: Optimizing Clinical Language Models for Predicting Patient Outcomes via Knowledge Distillation, Pruning, and Quantization. *International Journal of Medical Informatics* 195 (2025), 105764. doi:10.1016/j.ijmedinf.2024.105764

[50] Dan Hendrycks, Collin Burns, Anya Chen, and Spencer Ball. 2021. CUAD: An Expert-Annotated NLP Dataset for Legal Contract Review. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*. arXiv:2103.06268 https://arxiv.org/abs/2103.06268

[51] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. 2015. Distilling the Knowledge in a Neural Network. arXiv:1503.02531. arXiv:1503.02531 https://arxiv.org/abs/1503.02531

[52] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Jack W. Rae, Oriol Vinyals, and Laurent Sifre. 2022. Training Compute-Optimal Large Language Models. In *Advances in Neural Information Processing Systems*, Vol. 35. Curran Associates, Inc., New Orleans, Louisiana, USA. arXiv:2203.15556 doi:10.52202/068431-2176

[53] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *10th International Conference on Learning Representations (ICLR)*. OpenReview.net. https://openreview.net/forum?id=nZeVKeeFYf9

[54] Qitian Jason Hu, Jacob Bieker, Xiuyu Li, Nan Jiang, Benjamin Keigwin, Gaurav Ranganath, Kurt Keutzer, and Shriyash Kaustubh Upadhyay. 2024. RouterBench: A Benchmark for Multi-LLM Routing System. arXiv:2403.12031. arXiv:2403.12031 https://arxiv.org/abs/2403.12031

[55] Hugging Face. 2024. Open LLM Leaderboard: About. Leaderboard documentation (version 2). Accessed 3 October 2026. The leaderboard has since been retired. https://huggingface.co/docs/leaderboards/open_llm_leaderboard/about

[56] IBM Granite Team. 2025. IBM Granite 4.0: Hyper-Efficient, High-Performance Hybrid Models for Enterprise. https://www.ibm.com/new/announcements/ibm-granite-4-0-hyper-efficient-high-performance-hybrid-models.

[57] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*. Vienna, Austria. arXiv:2310.06770 https://arxiv.org/abs/2310.06770

[58] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. 2020. Scaling Laws for Neural Language Models. arXiv:2001.08361. arXiv:2001.08361 https://arxiv.org/abs/2001.08361

[59] Sayash Kapoor, Benedikt Stroebl, Peter Kirgis, Nitya Nadgir, Zachary S Siegel, Boyi Wei, Tianci Xue, Ziru Chen, Felix Chen, Saiteja Utpala, Franck Ndzomga, Dheeraj Oruganty, Sophie Luskin, Kangheng Liu, Botao Yu, Amit Arora, Dongyoon Hahm, Harsh Trivedi, Huan Sun, Juyong Lee, Tengjun Jin, Yifan Mai, Yifei Zhou, Yuxuan Zhu, Rishi Bommasani, Daniel Kang, Dawn Song, Peter Henderson, Yu Su, Percy Liang, and Arvind Narayanan. 2025. Holistic Agent Leaderboard: The Missing Infrastructure for AI Agent Evaluation. arXiv:2510.11977. arXiv:2510.11977 https://arxiv.org/abs/2510.11977

[60] Sayash Kapoor, Benedikt Stroebl, Zachary S. Siegel, Nitya Nadgir, and Arvind Narayanan. 2024. AI Agents That Matter. arXiv:2407.01502. arXiv:2407.01502 https://arxiv.org/abs/2407.01502

[61] Ishan Kavathekar, Raghav Donakanti, Ponnurangam Kumaraguru, and Karthik Vaidhyanathan. 2025. Small Models, Big Tasks: An Exploratory Empirical Study on Small Language Models for Function Calling. arXiv:2504.19277. arXiv:2504.19277 https://arxiv.org/abs/2504.19277

[62] Hyunjae Kim, Hyeon Hwang, Jiwoo Lee, Sihyeon Park, Dain Kim, Taewhoo Lee, Chanwoong Yoon, Jiwoong Sohn, Jungwoo Park, Olga Reykhart, Thomas Fetherston, Donghee Choi, Soo Heon Kwak, Qingyu Chen, and Jaewoo Kang. 2025. Small Language Models Learn Enhanced Reasoning Skills from Medical Textbooks. *npj Digital Medicine* 8

(2025), 240. doi:10.1038/s41746-025-01653-8

[63] Will Kurt. 2024. Say What You Mean: A Response to "Let Me Speak Freely". .txt engineering blog, https://blog.dottxt.ai/say-what-you-mean.html. Published 5 November 2024.

[64] Md Tahmid Rahman Laskar, Xue-Yong Fu, and Shashi Bhushan TN. 2026. SFT or RL for Tool-Calling Agents? A Controlled Study Across Data, Method, and Scale. arXiv:2609.17848. arXiv:2609.17848 https://arxiv.org/abs/2609.17848

[65] Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2023. Fast Inference from Transformers via Speculative Decoding. In *Proceedings of the 40th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 202)*. PMLR, Honolulu, Hawaii, USA, 19274–19286. https://proceedings.mlr.press/v202/leviathan23a.html

[66] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems*, Vol. 33. Curran Associates, Inc., 9459–9474. https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html

[67] Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. 2023. API-Bank: A Comprehensive Benchmark for Tool-Augmented LLMs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics, 3102–3116. doi:10.18653/v1/2023.emnlp-main.187

[68] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. 2024. EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty. arXiv:2401.15077. arXiv:2401.15077 https://arxiv.org/abs/2401.15077

[69] Percy Liang, Rishi Bommasani, Tony Lee, Dimitris Tsipras, Dilara Soylu, Michihiro Yasunaga, Yian Zhang, Deepak Narayanan, Yuhuai Wu, Ananya Kumar, Benjamin Newman, Binhang Yuan, Bobby Yan, Ce Zhang, Christian Cosgrove, Christopher D. Manning, Christopher Ré, Diana Acosta-Navas, Drew A. Hudson, et al. 2023. Holistic Evaluation of Language Models. *Transactions on Machine Learning Research* (2023). https://openreview.net/forum?id=iO4LZibEqW

[70] Opher Lieber, Barak Lenz, Hofit Bata, Gal Cohen, Jhonathan Osin, Itay Dalmedigos, Erez Safahi, Shaked Meirom, Yonatan Belinkov, Shai Shalev-Shwartz, Omri Abend, Raz Alon, Tomer Asida, Amir Bergman, Roman Glozman, Michael Gokhman, Avashalom Manevich, Nir Ratner, Noam Rozen, Erez Shwartz, Mor Zusman, and Yoav Shoham. 2024. Jamba: A Hybrid Transformer-Mamba Language Model. arXiv:2403.19887. arXiv:2403.19887 https://arxiv.org/abs/2403.19887

[71] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. 2024. AWQ: Activation-Aware Weight Quantization for On-Device LLM Compression and Acceleration. In *Proceedings of Machine Learning and Systems*, Vol. 6. MLSys, Santa Clara, California, USA. https://proceedings.mlsys.org/paper_files/paper/2024/hash/42a452cbafa9dd64e9ba4aa95cc1ef21-Abstract-Conference.html

[72] Qiqiang Lin, Muning Wen, Qiuying Peng, Guanyu Nie, Junwei Liao, Jun Wang, Xiaoyun Mo, Jiamu Zhou, Cheng Cheng, Yin Zhao, Jun Wang, and Weinan Zhang. 2024. Hammer: Robust Function-Calling for On-Device Language Models via Function Masking. arXiv:2410.04587. arXiv:2410.04587 https://arxiv.org/abs/2410.04587

[73] Weiwen Liu, Xu Huang, Xingshan Zeng, Xinlong Hao, Shuai Yu, Dexun Li, Shuai Wang, Weinan Gan, Zhengying Liu, Yuanqing Yu, Zezhong Wang, Yuxian Wang, Wu Ning, Yutai Hou, Bin Wang, Chuhan Wu, Xinzhi Wang, Yong Liu, Yasheng Wang, Duyu Tang, Dandan Tu, Lifeng Shang, Xin Jiang, Ruiming Tang, Defu Lian, Qun Liu, and Enhong Chen. 2024. ToolACE: Winning the Points of LLM Function Calling. arXiv:2409.00920. arXiv:2409.00920 https://arxiv.org/abs/2409.00920

[74] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. 2024. AgentBench: Evaluating LLMs as Agents. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*. Vienna, Austria. arXiv:2308.03688 https://arxiv.org/abs/2308.03688

[75] Zuxin Liu, Thai Hoang, Jianguo Zhang, Ming Zhu, Tian Lan, Shirley Kokane, Juntao Tan, Weiran Yao, Zhiwei Liu, Yihao Feng, Rithesh Murthy, Liangwei Yang, Silvio Savarese, Juan Carlos Niebles, Huan Wang, Shelby Heinecke, and Caiming Xiong. 2024. APIGen: Automated Pipeline for Generating Verifiable and Diverse Function-Calling Datasets. arXiv:2406.18518. arXiv:2406.18518 https://arxiv.org/abs/2406.18518

[76] Zechun Liu, Changsheng Zhao, Forrest Iandola, Chen Lai, Yuandong Tian, Igor Fedorov, Yunyang Xiong, Ernie Chang, Yangyang Shi, Raghuraman Krishnamoorthi, Liangzhen Lai, and Vikas Chandra. 2024. MobileLLM: Optimizing Sub-billion Parameter Language Models for On-Device Use Cases. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*. arXiv:2402.14905 https://arxiv.org/abs/2402.14905

[77] Llama Team, AI @ Meta. 2024. The Llama 3 Herd of Models. arXiv:2407.21783. https://arxiv.org/abs/2407.21783

[78] Anton Lozhkov, Raymond Li, Loubna Ben Allal, Federico Cassano, Joel Lamy-Poirier, Nouamane Tazi, Ao Tang, Dmytro Pykhtar, Jiawei Liu, Yuxiang Wei, Tianyang Liu, Max Tian, Denis Kocetkov, Arthur Zucker, Younes Belkada, Zijian Wang, Qian Liu, Dmitry Abulkhanov, Indraneil Paul, Zhuang Li, Wen-Ding Li, Megan Risdal, Jia Li, Jian Zhu, Terry Yue

Zhuo, Evgenii Zheltonozhskii, Nii Osae Osae Dade, Wenhao Yu, Lucas Krauß, Naman Jain, et al. 2024. StarCoder 2 and The Stack v2: The Next Generation. arXiv:2402.19173. arXiv:2402.19173 https://arxiv.org/abs/2402.19173

[79] Zhenyan Lu, Xiang Li, Dongqi Cai, Rongjie Yi, Fangming Liu, Xiwen Zhang, Nicholas D. Lane, and Mengwei Xu. 2024. Small Language Models: Survey, Measurements, and Insights. arXiv:2409.15790. arXiv:2409.15790 https://arxiv.org/abs/2409.15790

[80] Shuming Ma, Hongyu Wang, Lingxiao Ma, Lei Wang, Wenhui Wang, Shaohan Huang, Li Dong, Ruiping Wang, Jilong Xue, and Furu Wei. 2024. The Era of 1-bit LLMs: All Large Language Models are in 1.58 Bits. arXiv:2402.17764. arXiv:2402.17764 https://arxiv.org/abs/2402.17764

[81] Xinyin Ma, Gongfan Fang, and Xinchao Wang. 2023. LLM-Pruner: On the Structural Pruning of Large Language Models. arXiv:2305.11627. arXiv:2305.11627 https://arxiv.org/abs/2305.11627

[82] Xin Men, Mingyu Xu, Qingyu Zhang, Bingning Wang, Hongyu Lin, Yaojie Lu, Xianpei Han, and Weipeng Chen. 2024. ShortGPT: Layers in Large Language Models are More Redundant Than You Expect. arXiv:2403.03853. arXiv:2403.03853 https://arxiv.org/abs/2403.03853

[83] Meta AI. 2024. ExecuTorch: End-to-End Solution for Enabling On-Device AI. PyTorch Blog. https://pytorch.org/executorch/

[84] Meta AI. 2024. Llama 3.2 Model Card. https://github.com/meta-llama/llama-models/blob/main/models/llama3_2/MODEL_CARD.md. Retrieved 8 October 2026.

[85] Meta AI. 2024. Llama 3.2: Revolutionizing Edge AI and Vision with Open, Customizable Models. https://ai.meta.com/blog/llama-3-2-connect-2024-vision-edge-mobile-devices/. Published 25 September 2024.

[86] Grégoire Mialon, Clémentine Fourrier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. 2024. GAIA: A Benchmark for General AI Assistants. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*. Vienna, Austria. arXiv:2311.12983 https://arxiv.org/abs/2311.12983

[87] Microsoft. 2025. Phi-4-mini-instruct Model Card. https://huggingface.co/microsoft/Phi-4-mini-instruct. Retrieved 8 October 2026.

[88] MLC Team. 2024. MLC LLM: Universal LLM Deployment Engine. GitHub repository: https://github.com/mlc-ai/mlc-llm.

[89] Subhabrata Mukherjee, Arindam Mitra, Ganesh Jawahar, Sahaj Agarwal, Hamid Palangi, and Ahmed Awadallah. 2023. Orca: Progressive Learning from Complex Explanation Traces of GPT-4. arXiv:2306.02707. arXiv:2306.02707 https://arxiv.org/abs/2306.02707

[90] Saurav Muralidharan, Sharath Turuvekere Sreenivas, Raviraj Joshi, Marcin Chochowski, Mostofa Patwary, Mohammad Shoeybi, Bryan Catanzaro, Jan Kautz, and Pavlo Molchanov. 2024. Compact Language Models via Pruning and Knowledge Distillation. Advances in Neural Information Processing Systems (NeurIPS); arXiv:2407.14679. doi:10.52202/079017-1299

[91] Chien Van Nguyen, Xuan Shen, Ryan Aponte, Yu Xia, Samyadeep Basu, Zhengmian Hu, Jian Chen, Mihir Parmar, Sasidhar Kunapuli, Joe Barrow, Junda Wu, Ashish Singh, Yu Wang, Jiuxiang Gu, Franck Dernoncourt, Nesreen K. Ahmed, Nedim Lipka, Ruiyi Zhang, Xiang Chen, Tong Yu, Sungchul Kim, Hanieh Deilamsalehy, Namyong Park, Mike Rimer, Zhehao Zhang, Huanrui Yang, Ryan A. Rossi, and Thien Huu Nguyen. 2024. A Survey of Small Language Models. arXiv:2410.20011. arXiv:2410.20011 https://arxiv.org/abs/2410.20011

[92] NVIDIA. 2025. NVIDIA Nemotron Nano 2: An Accurate and Efficient Hybrid Mamba-Transformer Reasoning Model. arXiv:2508.14444. arXiv:2508.14444 https://arxiv.org/abs/2508.14444

[93] Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M. Waleed Kadous, and Ion Stoica. 2025. RouteLLM: Learning to Route LLMs with Preference Data. In *International Conference on Learning Representations (ICLR)*. arXiv:2406.18665 https://arxiv.org/abs/2406.18665

[94] OpenAI. 2023. GPT-4 Technical Report. arXiv:2303.08774. arXiv:2303.08774 https://arxiv.org/abs/2303.08774

[95] Kanghee Park, Jiayu Wang, Taylor Berg-Kirkpatrick, Nadia Polikarpova, and Loris D'Antoni. 2024. Grammar-Aligned Decoding. In *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:2405.21047 https://arxiv.org/abs/2405.21047

[96] Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. 2025. The Berkeley Function Calling Leaderboard (BFCL): From Tool Use to Agentic Evaluation of Large Language Models. In *Proceedings of the 42nd International Conference on Machine Learning (ICML) (Proceedings of Machine Learning Research, Vol. 267)*. PMLR. https://proceedings.mlr.press/v267/patil25a.html

[97] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. 2024. Gorilla: Large Language Model Connected with Massive APIs. In *Advances in Neural Information Processing Systems*. arXiv:2305.15334 doi:10.52202/079017-4020

[98] David Patterson, Joseph Gonzalez, Quoc Le, Chen Liang, Lluis-Miquel Munguia, Daniel Rothchild, David So, Maud Texier, and Jeff Dean. 2021. Carbon Emissions and Large Neural Network Training. arXiv:2104.10350. arXiv:2104.10350 https://arxiv.org/abs/2104.10350

[99] Akshara Prabhakar, Zuxin Liu, Ming Zhu, Jianguo Zhang, Tulika Awalgaonkar, Shiyu Wang, Zhiwei Liu, Haolin Chen, Thai Hoang, Juan Carlos Niebles, Shelby Heinecke, Weiran Yao, Huan Wang, Silvio Savarese, and Caiming Xiong. 2025. APIGen-MT: Agentic Pipeline for Multi-Turn Data Generation via Simulated Agent-Human Interplay. arXiv:2504.03601. arXiv:2504.03601 https://arxiv.org/abs/2504.03601

[100] Xiangyu Qi, Yi Zeng, Tinghao Xie, Pin-Yu Chen, Ruoxi Jia, Prateek Mittal, and Peter Henderson. 2023. Fine-tuning Aligned Language Models Compromises Safety, Even When Users Do Not Intend To! arXiv:2310.03693. https://arxiv.org/abs/2310.03693

[101] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2024. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-World APIs. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*. Vienna, Austria. arXiv:2307.16789 https://arxiv.org/abs/2307.16789

[102] Qwen Team, Alibaba Group. 2024. Qwen2.5 Technical Report. arXiv:2412.15115. arXiv:2412.15115 https://arxiv.org/abs/2412.15115

[103] Qwen Team, Alibaba Group. 2026. Qwen3.5 Small Models for On-Device Applications. https://qwen.ai/blog?id=qwen3.5. Alibaba; 0.8B–9B dense multimodal, Gated DeltaNet hybrid, released 2 March 2026.

[104] Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, Jérémy Rapin, Artyom Kozhevnikov, Ivan Evtimov, Joanna Bitton, Manish Bhatt, Cristian Canton Ferrer, Aaron Grattafiori, Wenhan Xiong, Alexandre Défossez, Jade Copet, Faisal Azhar, Hugo Touvron, Louis Martin, Nicolas Usunier, Thomas Scialom, and Gabriel Synnaeve. 2023. Code Llama: Open Foundation Models for Code. arXiv:2308.12950. arXiv:2308.12950 https://arxiv.org/abs/2308.12950

[105] Chenyang Shao, Xinyuan Hu, Yutang Lin, and Fengli Xu. 2025. Division-of-Thoughts: Harnessing Hybrid Language Model Synergy for Efficient On-Device Agents. In *Proceedings of the ACM Web Conference 2025*. arXiv:2502.04392 doi:10.1145/3696410.3714765

[106] Raghav Sharma and Manan Mehta. 2025. Small Language Models for Agentic Systems: A Survey of Architectures, Capabilities, and Deployment Trade offs. arXiv:2510.03847. arXiv:2510.03847 https://arxiv.org/abs/2510.03847

[107] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. 2017. Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer. In *5th International Conference on Learning Representations (ICLR)*. OpenReview.net, Toulon, France. https://openreview.net/forum?id=B1ckMDqlg

[108] Weizhou Shen, Chenliang Li, Hongzhan Chen, Ming Yan, Xiaojun Quan, Hehong Chen, Ji Zhang, and Fei Huang. 2024. Small LLMs Are Weak Tool Learners: A Multi-LLM Agent. arXiv:2401.07324. arXiv:2401.07324 https://arxiv.org/abs/2401.07324

[109] Sharath Turuvekere Sreenivas, Saurav Muralidharan, Raviraj Joshi, Marcin Chochowski, Ameya Sunil Mahabaleshwarkar, Gerald Shen, Jiaqi Zeng, Zijia Chen, Yoshi Suhara, Shizhe Diao, Chenhan Yu, Wei-Chun Chen, Hayley Ross, Oluwatobi Olabiyi, Ashwath Aithal, Oleksii Kuchaiev, Daniel Korzekwa, Pavlo Molchanov, Mostofa Patwary, Mohammad Shoeybi, Jan Kautz, and Bryan Catanzaro. 2024. LLM Pruning and Distillation in Practice: The Minitron Approach. arXiv:2408.11796. arXiv:2408.11796 https://arxiv.org/abs/2408.11796

[110] Gaurav Srivastava, Shuxiang Cao, and Xuan Wang. 2025. Towards Reasoning Ability of Small Language Models. arXiv:2502.11569. arXiv:2502.11569 https://arxiv.org/abs/2502.11569

[111] Mingjie Sun, Zhuang Liu, Anna Bair, and J. Zico Kolter. 2024. A Simple and Effective Pruning Approach for Large Language Models. In *12th International Conference on Learning Representations (ICLR)*. OpenReview.net, Vienna, Austria. https://openreview.net/forum?id=PxoFut3dWW

[112] Zhi Rui Tam, Cheng-Kuang Wu, Yi-Lin Tsai, Chieh-Yen Lin, Hung yi Lee, and Yun-Nung Chen. 2024. Let Me Speak Freely? A Study on the Impact of Format Restrictions on Performance of Large Language Models. arXiv:2408.02442. arXiv:2408.02442 https://arxiv.org/abs/2408.02442

[113] Technology Innovation Institute. 2025. Falcon-H1: A Family of Hybrid-Head Language Models Redefining Efficiency and Performance. arXiv:2507.22448. https://arxiv.org/abs/2507.22448

[114] Chunlin Tian, Dongqi Cai, Wanru Zhao, and Nicholas D. Lane. 2026. Beyond Scaling: Agents Are Heading to the Edge. arXiv:2605.18535. arXiv:2605.18535 https://arxiv.org/abs/2605.18535

[115] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is All You Need. In *Advances in Neural Information Processing Systems*, Vol. 30. Curran Associates, Inc., Long Beach, California, USA, 5998–6008. https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html

[116] Fali Wang, Zhiwei Zhang, Xianren Zhang, Zongyu Wu, Tzuhao Mo, Qiuhao Lu, Wanjing Wang, Rui Li, Junjie Xu, Xianfeng Tang, Qi He, Yao Ma, Ming Huang, and Suhang Wang. 2025. A Comprehensive Survey of Small Language Models in the Era of Large Language Models: Techniques, Enhancements, Applications, Collaboration with LLMs, and Trustworthiness. *ACM Transactions on Intelligent Systems and Technology* (2025). arXiv:2411.03350

doi:10.1145/3768165

[117] Jiacheng Wang, Yejun Zeng, Jinyang Guo, Yuqing Ma, Aishan Liu, and Xianglong Liu. 2025. SLMQuant: Benchmarking Small Language Model Quantization for Practical Deployment. arXiv:2511.13023. arXiv:2511.13023 doi:10.1145/3746262.3761973

[118] Brandon T. Willard and Rémi Louf. 2023. Efficient Guided Generation for Large Language Models. arXiv:2307.09702. https://arxiv.org/abs/2307.09702

[119] Mengzhou Xia, Tianyu Gao, Zhiyuan Zeng, and Danqi Chen. 2023. Sheared LLaMA: Accelerating Language Model Pre-training via Structured Pruning. arXiv:2310.06694. arXiv:2310.06694 https://arxiv.org/abs/2310.06694

[120] Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. 2022. SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models. arXiv:2211.10438. arXiv:2211.10438 https://arxiv.org/abs/2211.10438

[121] Haoran Xu, Baolin Peng, Hany Awadalla, Dongdong Chen, Yen-Chun Chen, Mei Gao, Young Jin Kim, Yunsheng Li, Liliang Ren, Yelong Shen, Shuohang Wang, Weijian Xu, Jianfeng Gao, and Weizhu Chen. 2025. Phi-4-Mini-Reasoning: Exploring the Limits of Small Reasoning Language Models in Math. arXiv:2504.21233. arXiv:2504.21233 https://arxiv.org/abs/2504.21233

[122] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, et al. 2025. Qwen3 Technical Report. arXiv:2505.09388. arXiv:2505.09388 https://arxiv.org/abs/2505.09388

[123] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. 2024. τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. arXiv:2406.12045. arXiv:2406.12045 https://arxiv.org/abs/2406.12045

[124] Jianguo Zhang, Tian Lan, Ming Zhu, Zuxin Liu, Thai Hoang, Shirley Kokane, Weiran Yao, Juntao Tan, Akshara Prabhakar, Haolin Chen, Zhiwei Liu, Yihao Feng, Tulika Awalgaonkar, Rithesh Murthy, Eric Hu, Zeyuan Chen, Ran Xu, Juan Carlos Niebles, Shelby Heinecke, Huan Wang, Silvio Savarese, and Caiming Xiong. 2025. xLAM: A Family of Large Action Models to Empower AI Agent Systems. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics (NAACL)*. arXiv:2409.03215 doi:10.18653/v1/2025.naacl-long.578

[125] Yue Zhang, Yafu Li, Leyang Cui, Deng Cai, Lemao Liu, Tingchen Fu, Xinting Huang, Enbo Zhao, Yu Zhang, Chen Xu, Yulong Chen, Longyue Wang, Anh Tuan Luu, Wei Bi, Freda Shi, and Shuming Shi. 2025. Siren's Song in the AI Ocean: A Survey on Hallucination in Large Language Models. *Computational Linguistics* (2025). doi:10.1162/coli.a.16

[126] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. 2024. WebArena: A Realistic Web Environment for Building Autonomous Agents. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*. Vienna, Austria. arXiv:2307.13854 https://arxiv.org/abs/2307.13854

[127] Zihan Zhou, Simon Kurz, and Zhixue Zhao. 2025. Revisiting Pruning vs Quantization for Small Language Models. In *Findings of the Association for Computational Linguistics: EMNLP 2025*. Association for Computational Linguistics, Suzhou, China, 12055–12070. doi:10.18653/v1/2025.findings-emnlp.645